

TEMA 8

DISEÑO RTL EN VHDL



ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba



DEPARTAMENTO DE
INGENIERÍA ELECTRÓNICA
Y DE COMPUTADORES
UNIVERSIDAD DE CÓRDOBA



Tema 1. Análisis de los circuitos integrados digitales



Tema 2. Diseño de sistemas digitales mediante Dispositivos Lógicos Programables



Tema 3. Introducción al lenguaje VHDL



Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL



Tema 5. Procesos y atributos en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL



Tema 8. Diseño RTL en VHDL



Tema 8. Diseño RTL en VHDL

❖ Objetivos

- Analizar la estructura general de un diseño RTL.
- Mostrar ejemplos de diseño de Rutas de Datos.
- Comprender la descripción en VHDL de las FSM+D.
- Asimilar la descripción en VHDL de las FSM+D.
- Analizar las ventajas e inconvenientes de las FSM+D frente a las FSM.

Tema 8. Diseño RTL en VHDL

❖ Bibliografía

- Enoch O. Hwang. Digital Logic and Microprocessor Design with VHDL. Ed. Delmar. 2005.
- Pong P. Chu. RTL Hardware Design Using VHDL. Ed. Wiley. 2006.
- Pong P. Chu. FPGA Prototyping by VHDL Examples. Ed. Wiley. 2008.
- Kevin Skahill. VHDL for Programmable logic. Ed. Addison-Wesley. 1996.

1. Introducción

- ❖ Para realizar una tarea compleja o simplemente para resolver un determinado problema, generalmente se describe el proceso mediante un **algoritmo**.
 - Secuencia detallada de acciones o pasos para realizar una tarea o resolver un problema.
- ❖ Los algoritmos generalmente se realizan por programas escritos en **lenguajes de programación tradicionales** (por software) que se ejecutan en un **procesador de propósito general**.
 - **Ventaja:**
 - ✓ Fácil descripción de los algoritmos.
 - **Inconveniente:**
 - ✓ Si el algoritmo requiere realizar operaciones específicas y complejas, y repetirlas un número muy elevado de veces, el tiempo de ejecución del programa por el procesador puede ser excesivo.
- ❖ Para mejorar el tiempo de ejecución del algoritmo es conveniente, y algunas veces imprescindible, implementarlo en **hardware**.
- ❖ Un algoritmo se implementa en hardware mediante la **metodología RT (Transferencia de Registros)**.
 - Se describe el funcionamiento del sistema por una secuencia de transferencias y procesamientos de datos entre los registros que lo componen.

1. Introducción

❖ Ejemplo de algoritmo.

- Hallar la raíz cuadrada entera de un número entero (X).

- ✓ Se aproxima el resultado por defecto.
- ✓ Se considera que se excluye el 0.

Input X

Resul = 0

Temp = 1

Resta = X

while (Resta – Temp) ≥ 0

```
{  Resta = Resta – Temp
  Resul = Resul + 1
  Temp = Temp + 2 }
```

output Resul

❖ Conclusiones:

- En el algoritmo o pseudocódigo se suelen usar variables para almacenar resultados intermedios de los cálculos: Temp y Resta.
 - ✓ Las variables corresponden a posiciones de memoria.
- ❖ La ejecución del algoritmo se realiza secuencialmente, y el orden de los pasos es importante.
 - ✓ Antes de hacer la resta se debe actualizar Temp.
- ❖ En VHDL las variables se usan sólo en los process, y la ejecución secuencial igualmente se hace mediante un process.
 - ✓ Aunque las descripciones con variables se pueden sintetizar en algunos casos, se convierten en señales.
 - ✓ No son interpretadas como posiciones de memoria, sino como registros.

1. Introducción

❖ Ejemplo de algoritmo.

- Hallar la raíz cuadrada entera de un número entero (X).

- ✓ Se aproxima el resultado por defecto.
- ✓ Se considera que se excluye el 0.

Input X

Resul = 0

Temp = 1

Resta = X

while (Resta – Temp) ≥ 0

```
{  Resta = Resta – Temp  
  Resul = Resul + 1  
  Temp = Temp + 2 }
```

output Resul

❖ Conclusiones:

- En el algoritmo o pseudocódigo se suelen usar variables para almacenar resultados intermedios de los cálculos: Temp y Resta.
 - ✓ Las variables corresponden a posiciones de memoria.
- ❖ La ejecución del algoritmo se realiza secuencialmente, y el orden de los pasos es importante.
 - ✓ Antes de hacer la resta se debe actualizar Temp.
- ❖ En VHDL las variables se usan sólo en los process, y la ejecución secuencial igualmente se hace mediante un process.
 - ✓ Aunque las descripciones con variables se pueden sintetizar en algunos casos, se convierten en señales.
 - ✓ No son interpretadas como posiciones de memoria, sino como registros.

1. Introducción

❖ Ejemplo de cálculo de la raíz cuadrada entera de un número entero, X.

❖ Ejemplo de algoritmo.

- Hallar la raíz cuadrada entera de un número entero (X).
 - ✓ Se aproxima el resultado por defecto.
 - ✓ Se considera que se excluye el 0.
- Input X
Resul = 0
Temp = 1 Resta = X
while (Resta – Temp) ≥ 0
{ Resta = Resta – Temp
 Resul = Resul + 1
 Temp = Temp + 2 }
output Resul

• X = 5

	Antes			Después		
i	Resta	Temp	Resul	Resta	Temp	Resul
1	5	1	0	4	3	1
2	4	3	1	1	5	2
3	1	5	2			2

1. Introducción

❖ Ejemplo de cálculo de la raíz cuadrada entera de un número entero, X.

❖ Ejemplo de algoritmo.

- Hallar la raíz cuadrada entera de un número entero (X).

- ✓ Se aproxima el resultado por defecto.
- ✓ Se considera que se excluye el 0.

Input X

Resul = 0

Temp = 1 Resta = X

while (Resta – Temp) ≥ 0

{ Resta = Resta – Temp

 Resul = Resul + 1

 Temp = Temp + 2 }

output Resul

• X = 9

	Antes			Después		
i	Resta	Temp	Resul	Resta	Temp	Resul
1	9	1	0	8	3	1
2	8	3	1	5	5	2
3	5	5	2	0	7	3
4	0	7	3			3

2. Metodología RT

*Para realizar un **algoritmo** en hardware se debe:*

Implementar las variables usadas en el algoritmo.

Realizar la secuencia de operaciones con las variables definidas en el algoritmo.

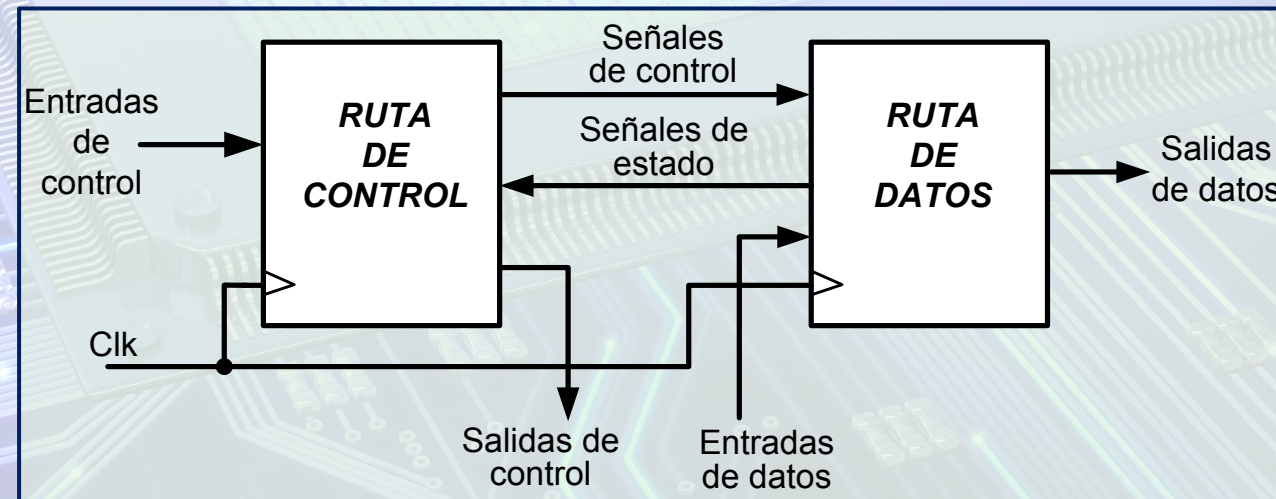
*Se realiza mediante **la metodología RT (Transferencia de Registros)**.*

Características básicas de un sistema diseñado por la metodología RT:

Consta de dos bloques o unidades:

Ruta de Datos:

Ruta de control:



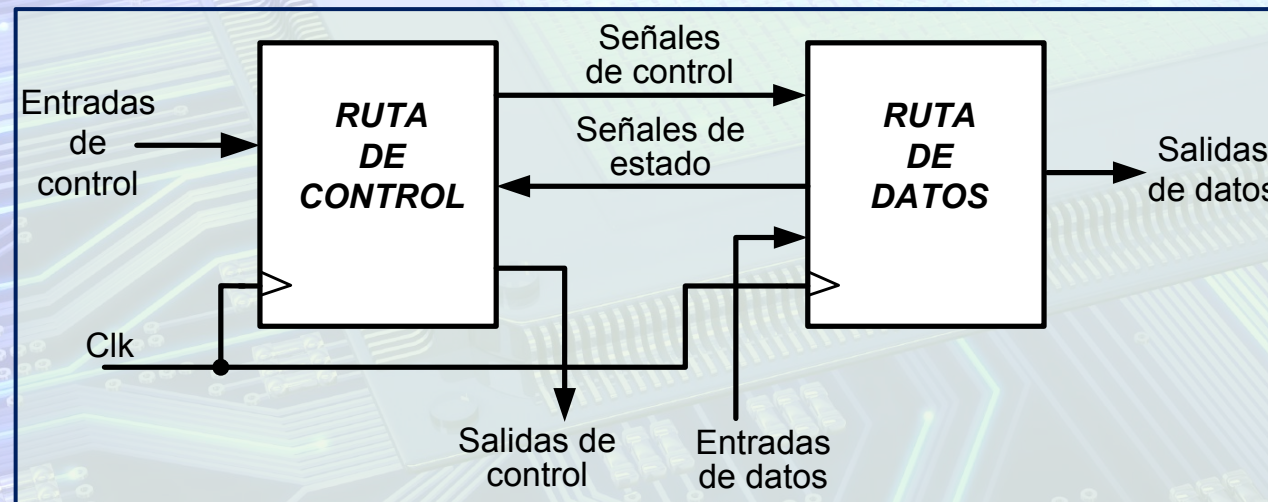
2. Metodología RT

❖ Ruta de Datos:

- Usa **registros** para almacenar los datos. Corresponden a las variables empleadas en el algoritmo.
- Realiza las **operaciones** indicadas en el algoritmo.

❖ Ruta de Control:

- Genera las **señales de control** a la **Ruta de Datos** de forma que se realicen las operaciones con los registros en la secuencia indicada por el algoritmo.



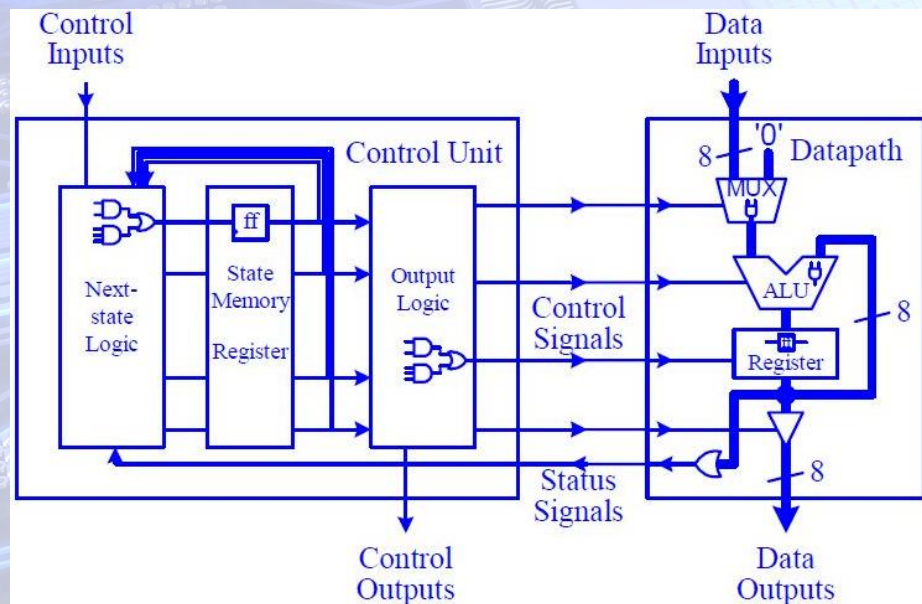
- La estructura hardware usada en la metodología RT coincide con la de un procesador.
 - ✓ Por implementar un algoritmo concreto se trata de un procesador específico.
 - ✓ Si se integra dentro de una FPGA se denomina procesador embebido.

2. Metodología RT

❖ Análisis de la arquitectura de un procesador específico

▪ Ruta de datos

- ✓ Realiza todas las operaciones que se deben hacer con los datos.
- ✓ **Componentes** de los que consta:
 - **Unidades funcionales** como sumadores, desplazadores, multiplicadores, ALUs y comparadores.
 - **Registros** y otros **elementos de memoria** para el almacenamiento temporal de los datos y procesamiento de los mismos.
 - **Buses y multiplexores** para la transferencia de datos entre los diferentes componentes en la ruta de datos.



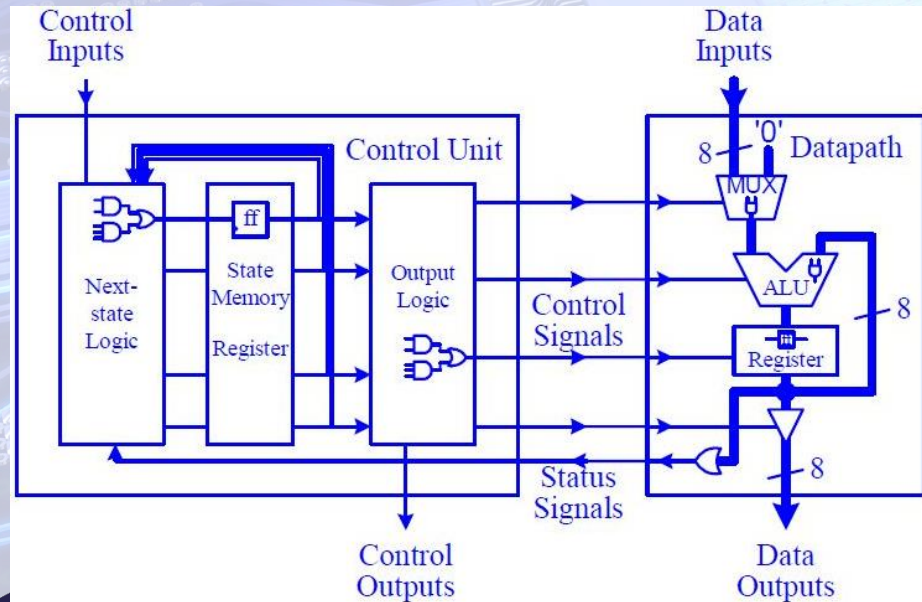
- ✓ Los registros realizan una serie de operaciones elementales que generalmente consisten en transferir el contenido de un registro a otro, procesando o no el contenido del registro origen.
- ✓ Estas operaciones elementales se denominan microoperaciones, y se ejecutan en un solo ciclo de reloj.
- ✓ Los datos externos pueden entrar a la ruta de datos a través de las líneas de entrada de datos.
- ✓ Los resultados de la computación se pueden devolver a través de las líneas de salida de datos.

2. Metodología RT

❖ Análisis de la arquitectura de un procesador específico (2).

▪ Ruta o unidad de control o controlador.

- ✓ Genera las señales de control a la ruta de datos de forma que ésta realice la secuencia de microoperaciones necesaria para implementar el algoritmo.
- La unidad de control en un procesador embebido en una FPGA se diseña como una **FSM**.
 - La **memoria de estado**, que es un registro, indica el **estado actual** en el que se encuentra la FSM.
 - La **lógica de estado siguiente** determina el estado al que evoluciona la FSM en el próximo flanco de reloj dependiendo del estado actual en el que se encuentra, y del valor de las entradas de control y las señales de estado.



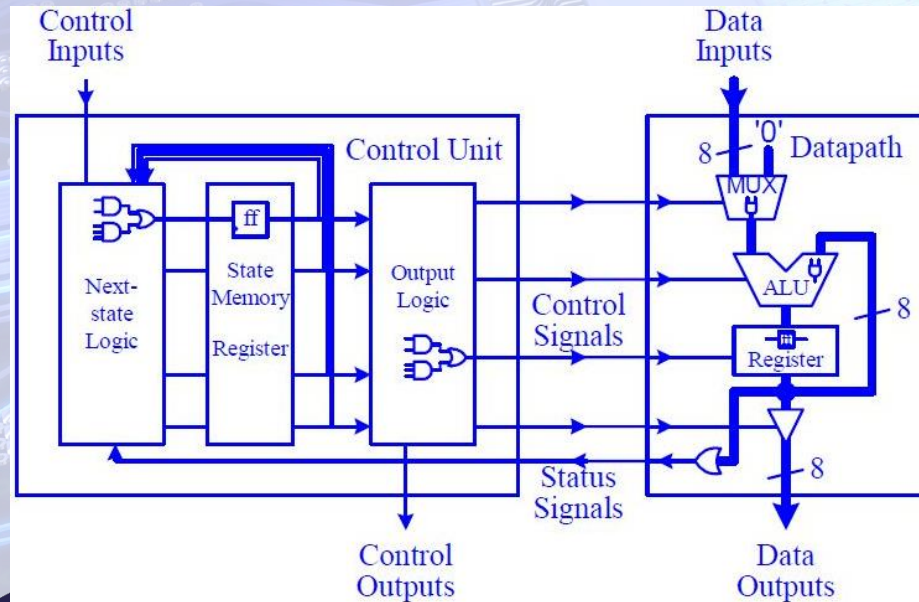
- En cada estado, la lógica de salida genera todas las señales de control que controlan la ruta de datos.
- **La ruta de datos genera señales de estado** a la unidad de control que determinan el estado siguiente de la FSM.
 - Indican el estado de las operaciones realizadas.
 - Pueden coincidir con la salida de los comparadores. Por ejemplo, una señal Z que se pone a 1 si el resultado es 0.
 - La salida de acarreo de un sumador.
 - El bit de signo, si se opera con números con signo representados, por ejemplo, en el sistema de complemento a 2.
- Al terminar las operaciones se puede activar una **salida de control** para indicar a otro dispositivo que es válido el dato de salida.

2. Metodología RT

❖ Análisis de la arquitectura de un procesador específico (2).

▪ Ruta o unidad de control o controlador.

- ✓ Genera las señales de control a la ruta de datos de forma que ésta realice la secuencia de microoperaciones necesaria para implementar el algoritmo.
- La unidad de control en un procesador embebido en una FPGA se diseña como una **FSM**.
 - La **memoria de estado**, que es un registro, indica el **estado actual** en el que se encuentra la FSM.
 - La **lógica de estado siguiente** determina el estado al que evoluciona la FSM en el próximo flanco de reloj dependiendo del estado actual en el que se encuentra, y del valor de las entradas de control y las señales de estado.



- En cada estado, la lógica de salida genera todas las señales de control que controlan la ruta de datos.
- **La ruta de datos genera señales de estado** a la unidad de control que determinan el estado siguiente de la FSM.
 - Indican el estado de las operaciones realizadas.
 - Pueden coincidir con la salida de los comparadores. Por ejemplo, una señal Z que se pone a 1 si el resultado es 0.
 - La salida de acarreo de un sumador.
 - El bit de signo, si se opera con números con signo representados, por ejemplo, en el sistema de complemento a 2.
- Al terminar las operaciones se puede activar una **salida de control** para indicar a otro dispositivo que es válido el dato de salida.

3. Descripción VHDL de procesadores específicos o sistemas diseñados por la Metodología RT.

❖ Alternativas de descripción en VHDL

▪ Máquina de Estados Finita y Ruta de Datos (FSM+D)

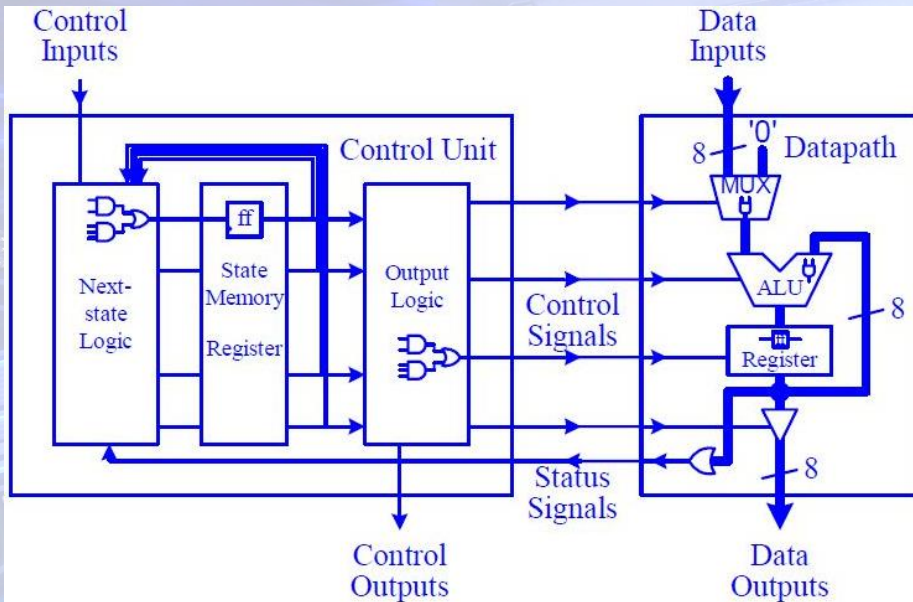
✓ Se describe como un sistema jerárquico de dos niveles.

- En el **nivel inferior** se describe por separado en VHDL:
 - La Ruta de Datos mediante los estilos **behavioral** y/o **data flow**.
 - La Ruta de Control que consiste en una **FSM**.
- En el **nivel raíz** se conecta la Ruta de Datos y la FSM usando las señales de control y de estado mediante el estilo **estructural**.

▪ Máquina de Estados Finita con Ruta de Datos (FSMD)

✓ La descripción se realiza mediante una sola entidad.

- Se describe la FSM usando el **estilo behavioral** de VHDL.
- Todas las **operaciones de la ruta de datos** se incluyen en la descripción de la FSM usando los operadores correspondientes de VHDL.
 - La herramienta de síntesis genera automáticamente la FSM y la ruta de datos, y los conecta mediante las señales de control y de estado.



3. Descripción VHDL de procesadores específicos o sistemas diseñados por la Metodología RT.

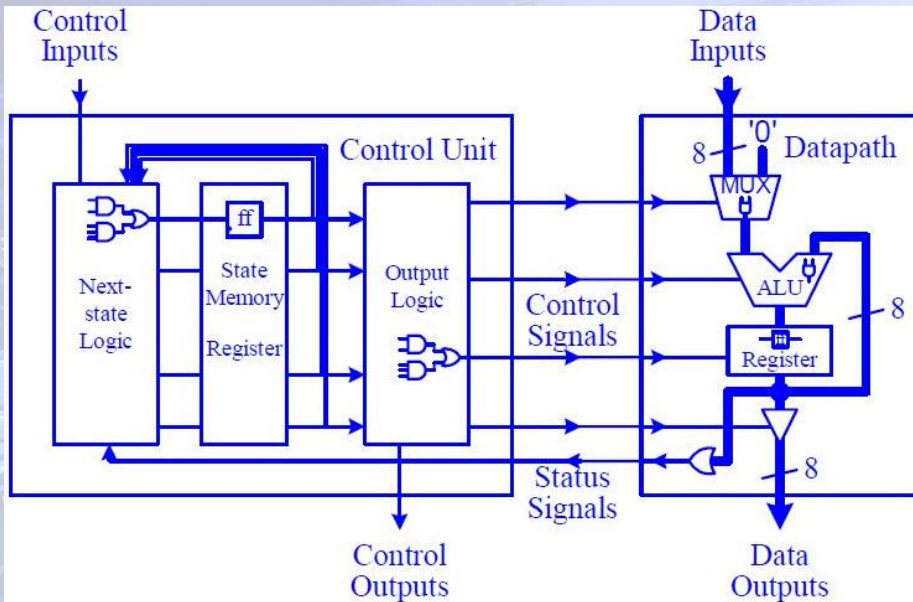
❖ Comparativa. FSM+D versus FSMD.

■ Ventajas de la FSM+D

- ✓ El diseñador puede describir de forma más optimizada todos los componentes del sistema.
- ✓ Se mejora las prestaciones del sistema:
 - Mínimo número posible de recursos lógicos usados.
 - Máxima frecuencia de funcionamiento.

■ Ventajas de la FSMD

- ✓ Se facilita la descripción del diseño, ya que se usa un solo fichero VHDL y se simplifica la descripción de la ruta de datos.
- ✓ El diseñador tiene completo control sobre la temporización de la FSM, indicando los estados o los ciclos de reloj en los que se ejecuta cada una de las microoperaciones.

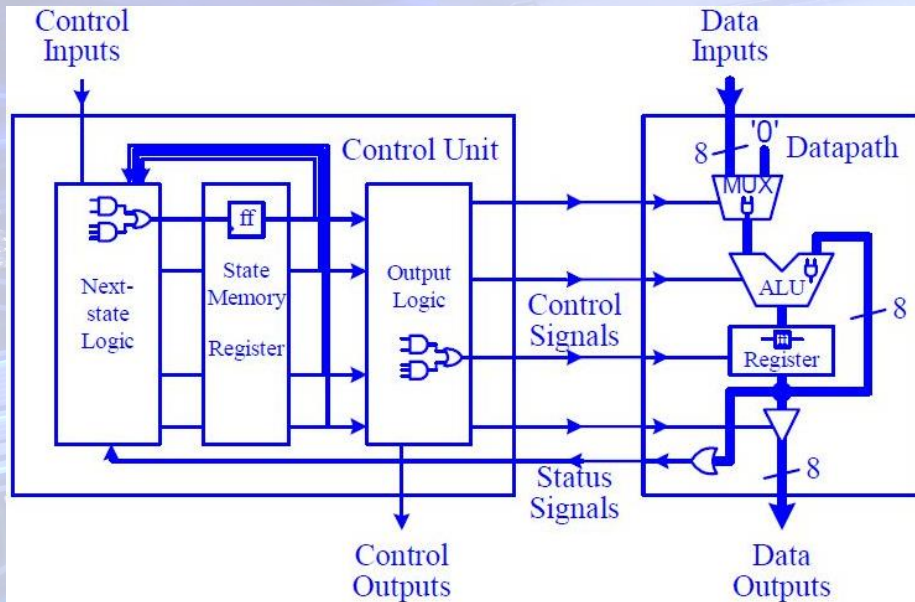


4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Diseño de rutas de datos específicas (1).

- El objetivo para diseñar una ruta de datos específica es obtener un circuito que resuelva un problema concreto.
- Se parte del algoritmo que resuelve el problema y, de éste, se diseña el circuito correspondiente en el nivel de transferencia de registros.

- ✓ El diseñador debe determinar **las microoperaciones necesarias para realizar el algoritmo**, y como se transfieren los datos entre los registros a través de las unidades funcionales.
- ✓ En el **proceso de diseño** debe decidirse:
 - ¿Qué **tipos de registros** hay que usar y **cuántos** son necesarios?
 - ¿Qué **operaciones** debe realizar cada registro?
 - ¿Qué **tipos de unidades funcionales** hay que usar y **cuántas** son necesarias?
 - ¿Puede una unidad funcional **compartirse** entre dos o más operaciones?
 - ¿Cuántos registros y unidades funcionales **se conectan juntas** de forma que se realicen **todas las transferencias de datos** especificadas por el algoritmo?



4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Ejemplo 1.

▪ Ruta de datos para realizar la microoperación: $A \leftarrow A + 3$

- Al valor de la variable A se le suma 3 y el resultado se vuelve a almacenar en la misma variable A.

• Componentes necesarios:

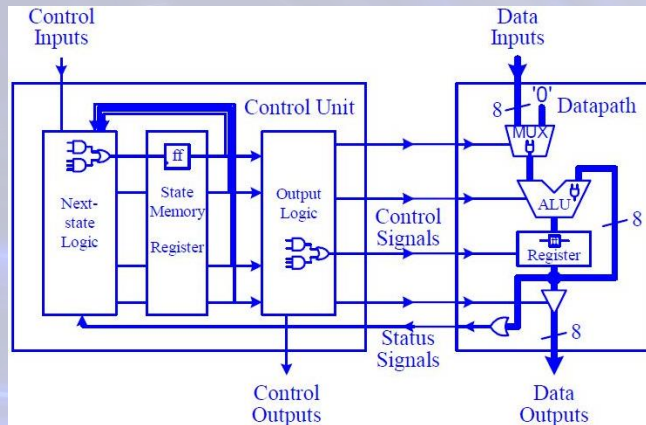
- ✓ Un registro para almacenar la variable A.
 - El registro debe realizar la operación de carga paralela.
- ✓ Un sumador para sumar 3 a la variable A.

• Terminales de control:

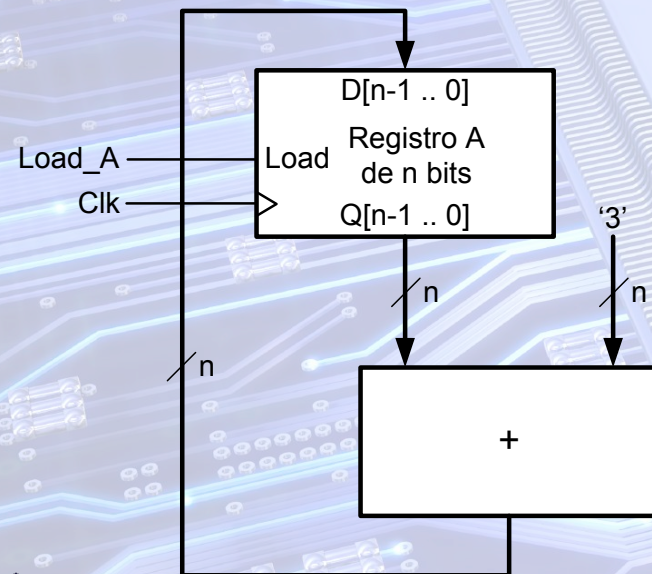
- ✓ El registro debe tener un terminal de control (Load) que habilite la operación de carga cuando tenga que realizarse la suma.

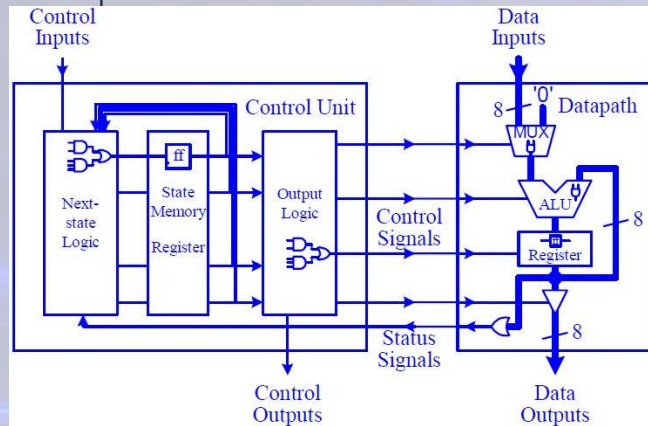
• Conexión de los componentes:

- ✓ Se ha considerado que la variable A es de n bits.
- ✓ Load_A es un ejemplo de señal de control de las rutas de datos.
 - Al activarse el registro A almacena el resultado de la suma.
 - La Unidad de Control debe activar Load_A cuando tenga que realizarse esta operación.



Ruta de Datos necesaria





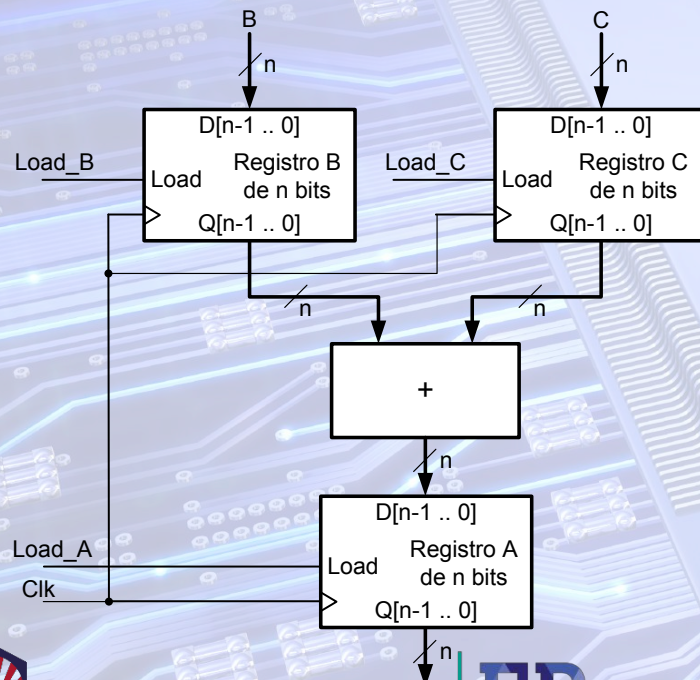
4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

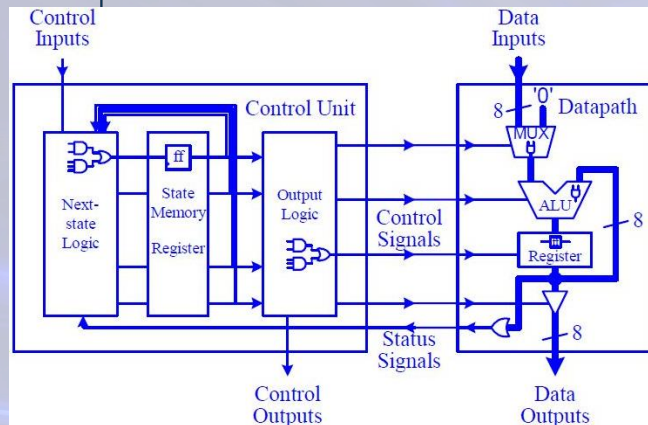
❖ Ejemplo 2.

▪ Ruta de datos para realizar la microoperación:

$$A \leftarrow B + C$$

- La suma de las variables B y C se almacena en la variable A.
- **Componentes necesarios:**
 - ✓ Tres registros de n bits para almacenar las variables A, B y C.
 - ✓ Un sumador de n bits.
- **Terminales de control:**
 - ✓ El registro A debe tener un terminal de control (Load) que habilite la operación de carga cuando tenga que realizarse la suma.
 - ✓ Los registros B y C deberán tener también un terminal de control para cargar las variables a sumar.
- **Conexión de los componentes:**
 - ✓ Se ha considerado que las variables y por tanto los registros son de n bits.
 - ✓ Las señales de control son **Load_A, Load_B y Load_C**.
 - Al activarse el terminal de control de un registro, éste almacena el dato que haya en su entrada.
 - ✓ Todos los registros usan **la misma señal de reloj**. Esto es importante, ya que el diseño debe ser **completamente síncrono**.





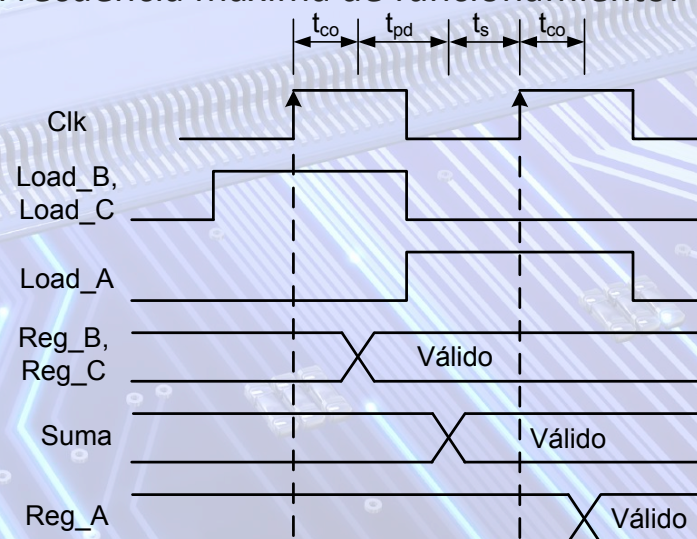
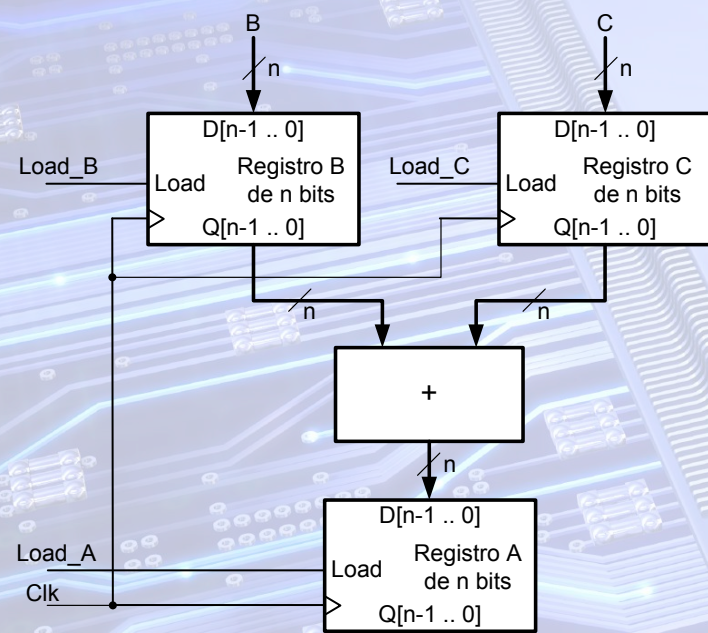
4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

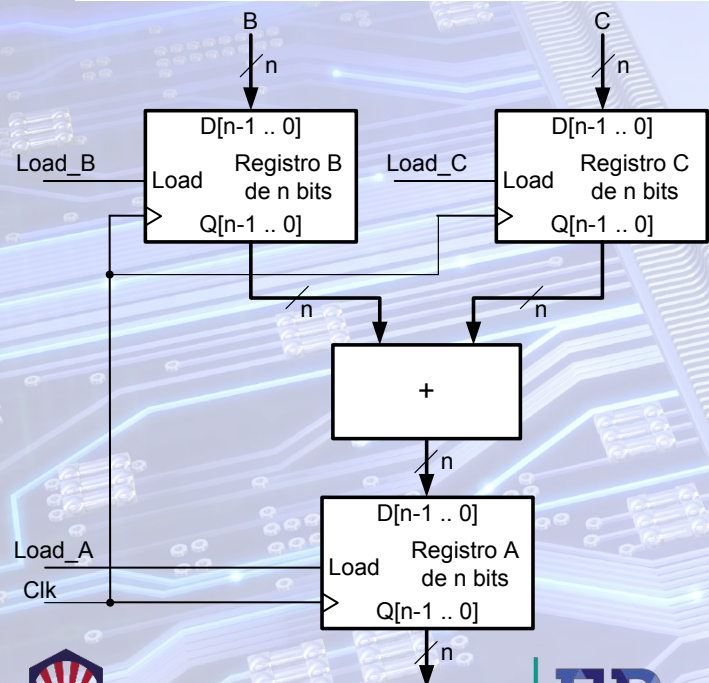
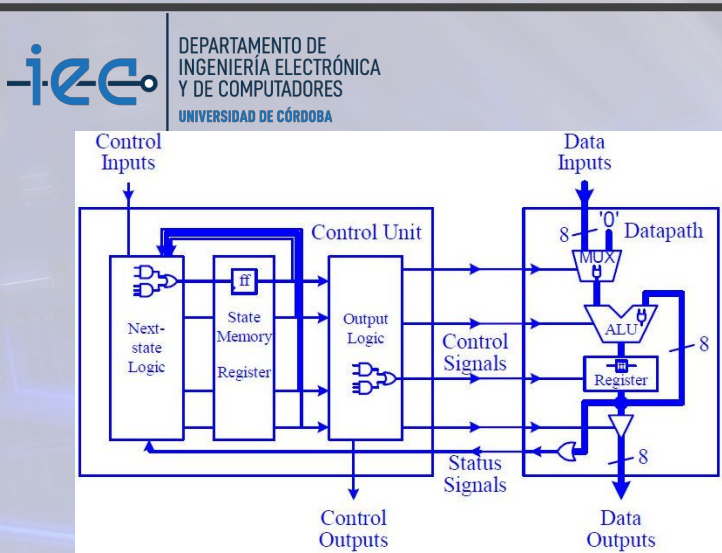
❖ Ejemplo 2.

▪ Ruta de datos para realizar la operación $A \leftarrow B + C$

✓ Funcionamiento.

- La unidad de control activará primero Load_B y Load_C para cargar las variables B y C en los correspondientes registros.
- En el siguiente ciclo de reloj activará Load_A para almacenar la suma.
- El sumador realiza la suma de B y C entre ambos flancos.
- Frecuencia máxima de funcionamiento:





4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

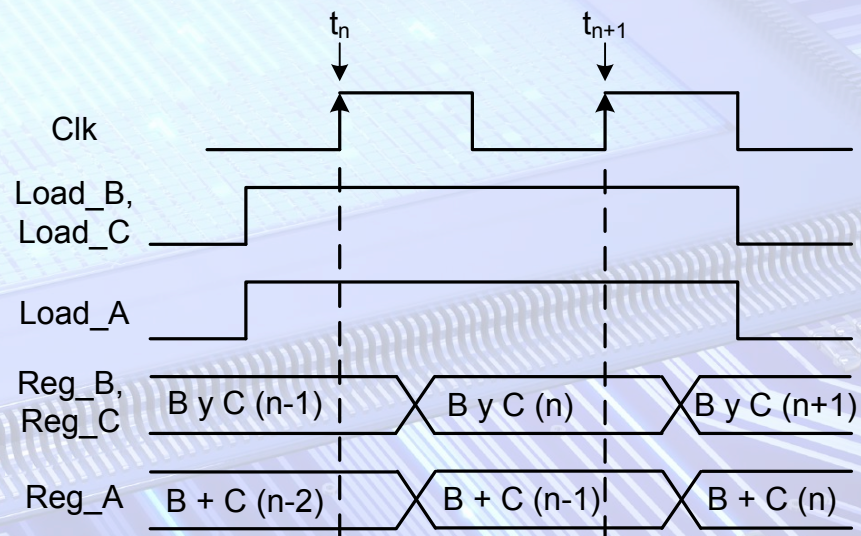
❖ Ejemplo 2.

▪ Ruta de datos para realizar la microoperación $A \leftarrow B + C$

• Funcionamiento.

✓ ¿Se puede activar al mismo tiempo todas las señales de control de los registros?

✓ **Cronograma.**



- ✓ Es posible ya que son registros independientes.
- ✓ Sólo hay que tener en cuenta la dependencia de los datos.
- ✓ Se obtiene una ruta de datos del tipo pipeline.

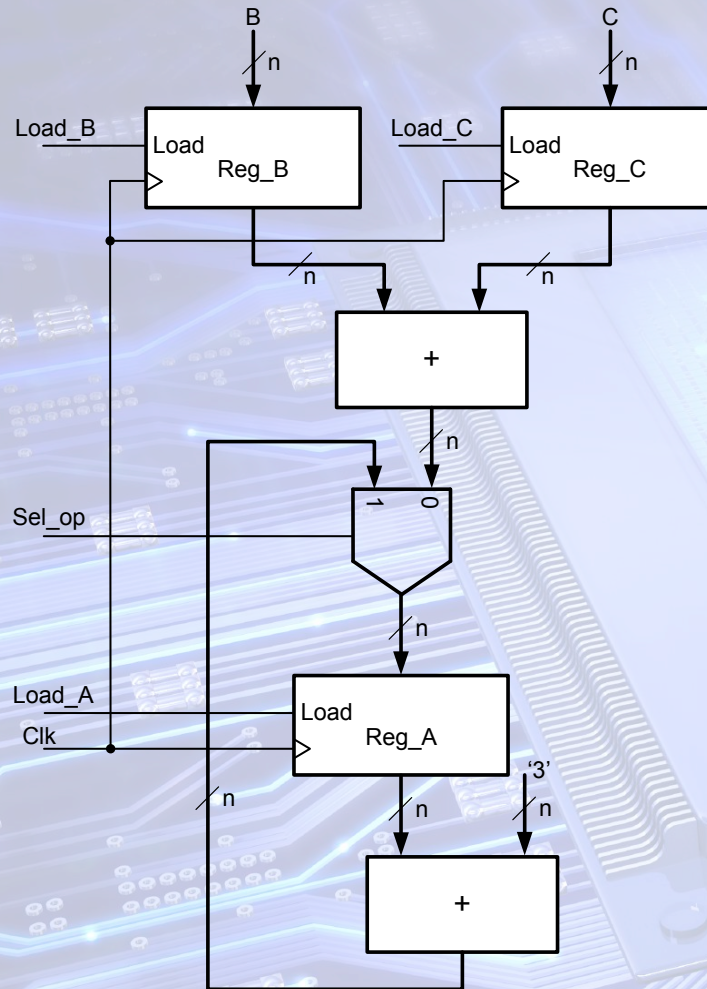
4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Ejemplo 3.

- Ruta de datos para realizar una de las dos microoperaciones de los ejemplos anteriores:

$$A \leftarrow B + C \quad \text{y} \quad A \leftarrow A + 3$$

- Se debe unir las dos rutas de datos en una.
- **Componentes necesarios:**
 - ✓ Tres registros para almacenar las variables A, B y C.
 - ✓ Se usarán dos sumadores.
 - Cada sumador realiza una operación.
 - ✓ El registro A tiene dos fuentes: $B + C$, y $A + 3$.
 - Por ello, se usa un multiplexor de 2 a 1 para seleccionar cuál de las dos fuentes se almacena en A.



4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Ejemplo 3.

- Ruta de datos para realizar una de las dos microoperaciones de los ejemplos anteriores:

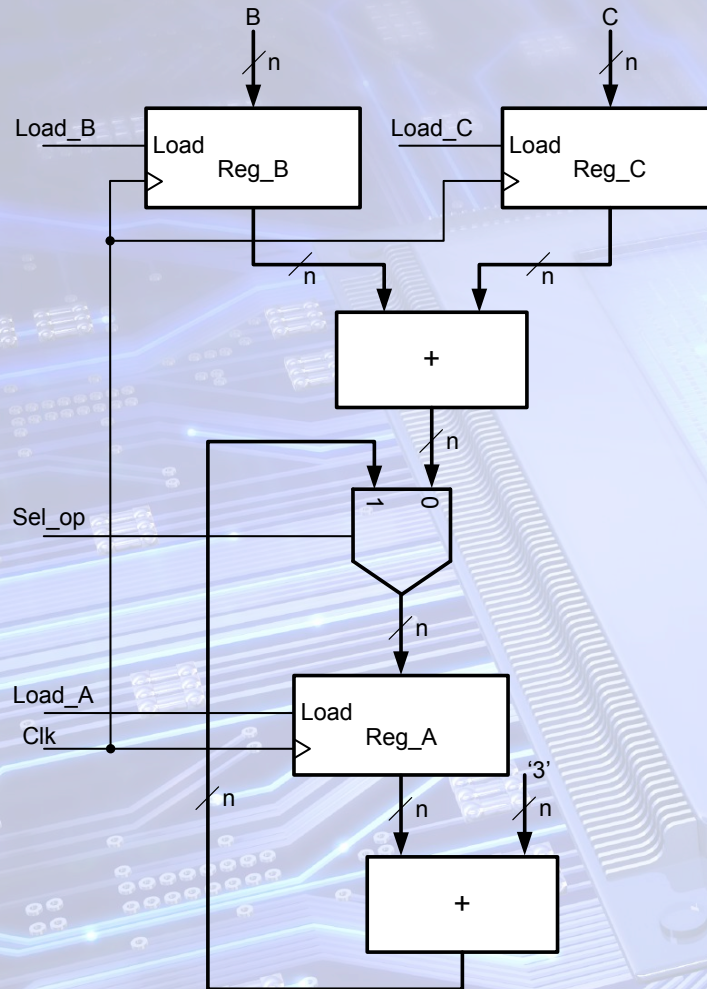
$$A \leftarrow B + C \quad \text{y} \quad A \leftarrow A + 3$$

- **Terminales de control:**

- ✓ Todos los registros deben tener un terminal de control (Load) que habilite la operación de carga.
- ✓ La entrada de control de selección del multiplexor debe controlarse mediante una señal de control (Sel_op) para seleccionar qué operación se va a realizar.

- **Conexión de los componentes:**

- ✓ Las salidas de los sumadores se conectan a las entradas de datos del mux, y la salida de éste a la entrada del registro A.



4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Ejemplo 3.

- Ruta de datos para realizar una de las dos microoperaciones de los ejemplos anteriores:

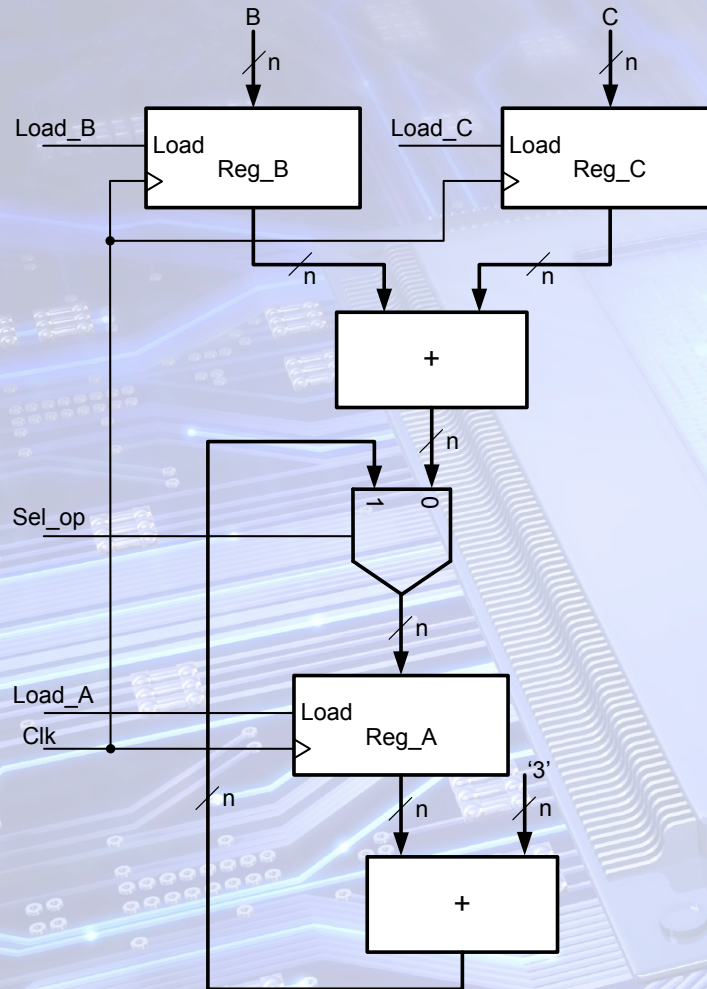
$$A \leftarrow B + C \quad \text{y} \quad A \leftarrow A + 3$$

- **Terminales de control:**

- ✓ Todos los registros deben tener un terminal de control (Load) que habilite la operación de carga.
- ✓ La entrada de control de selección del multiplexor debe controlarse mediante una señal de control (Sel_op) para seleccionar qué operación se va a realizar.

- **Conexión de los componentes:**

- ✓ Las salidas de los sumadores se conectan a las entradas de datos del mux, y la salida de éste a la entrada del registro A.



4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

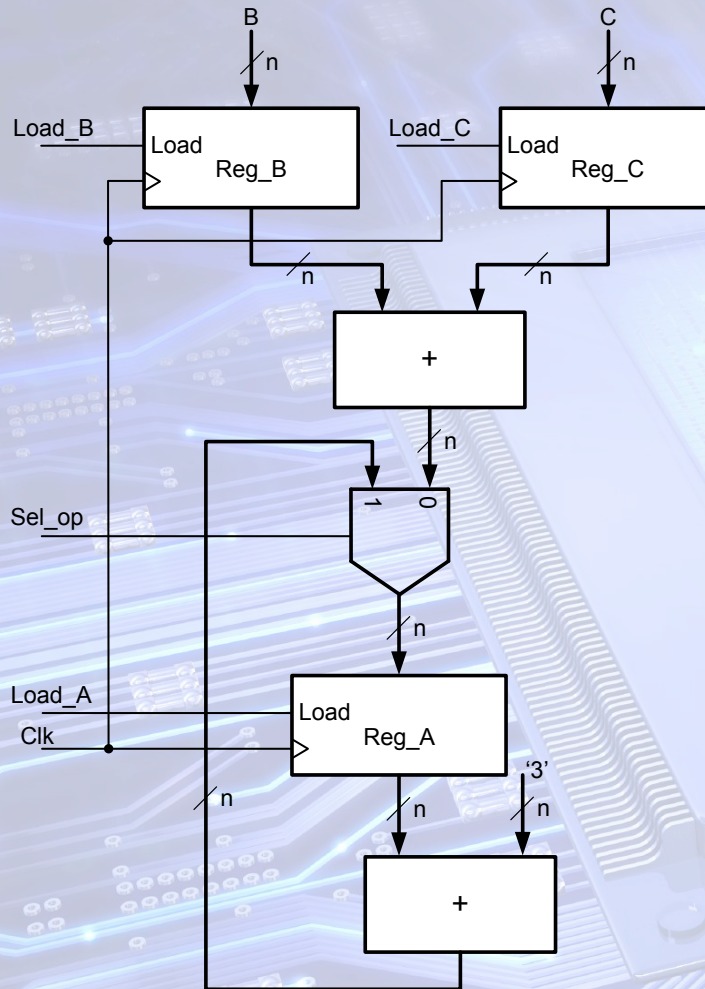
❖ Ejemplo 3.

- Ruta de datos para realizar una de las dos operaciones de los ejemplos anteriores:

$$A \leftarrow B + C \quad \text{y} \quad A \leftarrow A + 3$$

✓ Funcionamiento.

- El multiplexor selecciona qué suma se almacena en el registro A, y por tanto la microoperación que realiza la ruta de datos.
 - $\text{Sel_op} = 0 \quad \Rightarrow \quad A \leftarrow B + C$
 - $\text{Sel_op} = 1 \quad \Rightarrow \quad A \leftarrow A + 3$
- ¿Se pueden realizar al mismo tiempo, es decir en el mismo flanco de reloj, ambas operaciones?
 - Evidentemente no, ya que que el resultado de ambas operaciones se almacenan en el mismo registro (A), y éste sólo puede almacenar un dato en un flanco de reloj.
 - Se necesitarán dos flancos de reloj realizando una operación diferente en cada uno de ellos.



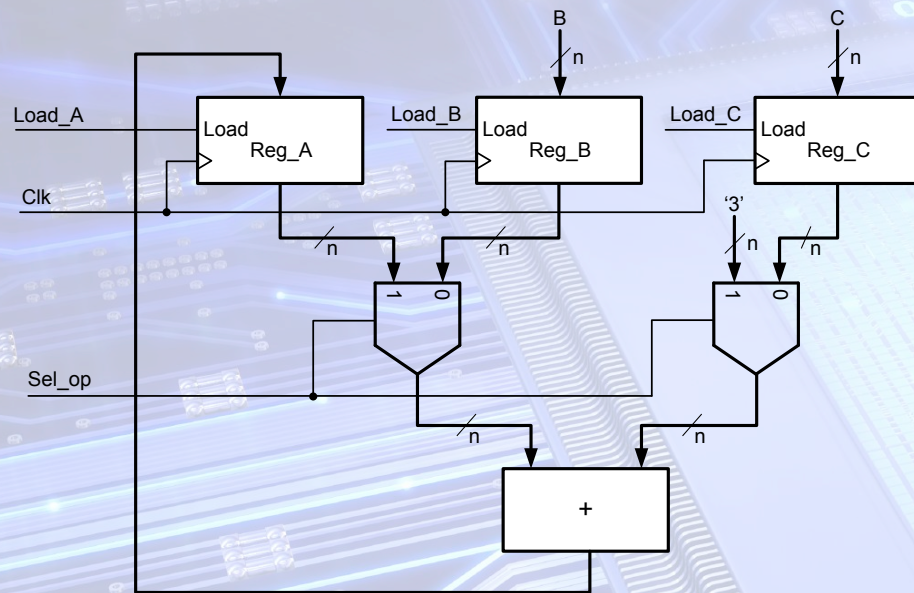
4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Ejemplo 4.

- Ruta de datos para realizar una de las dos microoperaciones de los ejemplos anteriores usando un solo sumador.

$$A \leftarrow B + C \quad \text{y} \quad A \leftarrow A + 3$$

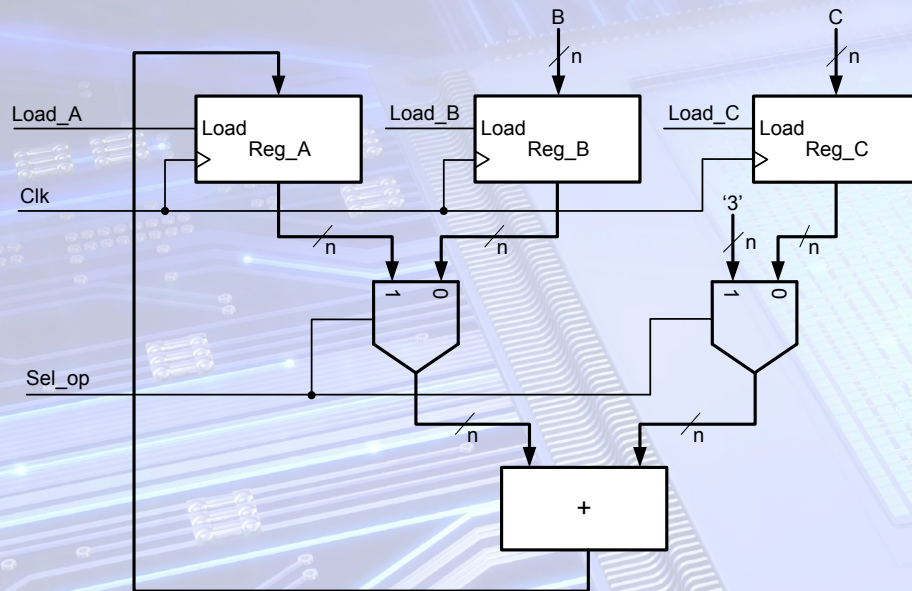
- Al usarse un solo sumador cada entrada del sumador tiene dos fuentes posibles:
 - ✓ 1ª Entrada del sumador: B o A
 - ✓ 2ª Entrada del sumador: C o 3
- Se debe usar 2 multiplexores.
 - ✓ Cada mux selecciona para una entrada del sumador la fuente correspondiente a la operación a realizar.



4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Ejemplo 4 (continuación).

- Ruta de datos para realizar una de las dos microoperaciones de los ejemplos anteriores usando un solo sumador.



$$A \leftarrow B + C \quad \text{y} \quad A \leftarrow A + 3$$

✓ Componentes necesarios:

- Tres registros para almacenar las variables A, B y C.
- Dos multiplexores de 2 a 1 de n bits.
- Un sumador de n bits.

✓ Terminales de control:

- Todos los registros deben tener un terminal de control (Load) que habilite la operación de carga.
- La entrada de control de selección de ambos multiplexores se conectará a una señal de control (Sel_op) para seleccionar los operandos según la microoperación que se vaya a realizar.

- Sel_op = 0 ⇔ $A \leftarrow B + C$
- Sel_op = 1 ⇔ $A \leftarrow A + 3$

4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

❖ Selección de Unidades Funcionales (1).

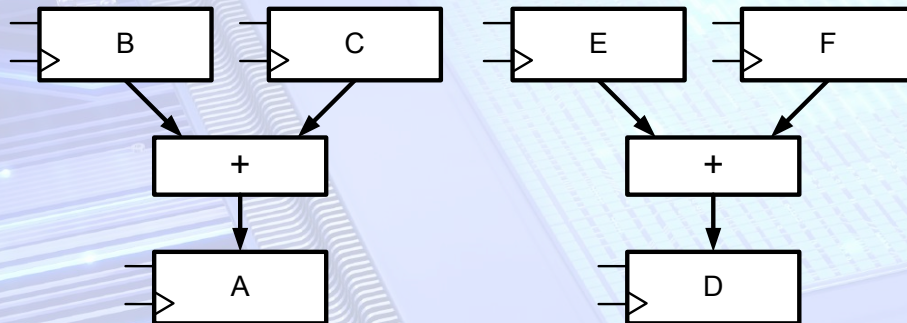
- La selección de las unidades funcionales (sumador, sumador/restador, multiplicador o ALU, entre otras) depende de las operaciones realizadas en el algoritmo.
 - Si en el algoritmo se realiza la suma y resta de variables:
 - ✓ Se empleará un sumador/restador.
 - Si sólo se realiza la suma:
 - ✓ Se empleará un sumador o varios según los requisitos del sistema.
- El uso de un **solo sumador puede reducir el número de componentes** de la ruta de datos, pero **aumentará su complejidad**, ya que hay un mayor número de conexiones entre los componentes.
 - Ejemplo. El algoritmo contiene las operaciones:
 - ✓ $A = B + C$
 - ✓ $D = E + F$

4. Descripción FSM+D en VHDL. Diseño de Rutas de Datos Específicas.

• Ejemplo.

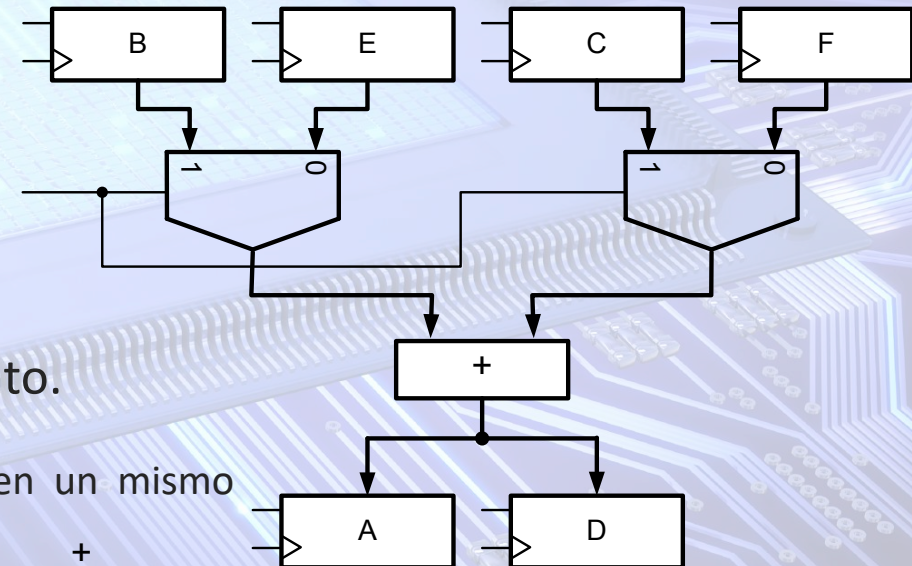
Microoperaciones:

- $A \leftarrow B + C$
- $D \leftarrow E + F$



• Selección de Unidades Funcionales (2).

- Dos opciones:
 - ✓ Dos sumadores.
 - ✓ Un solo sumador.



• En términos de velocidad de funcionamiento.

- Dos sumadores:
 - ✓ Se pueden ejecutar ambas operaciones en un mismo flanco de reloj.
- Un solo sumador:
 - ✓ Se tienen que ejecutar ambas operaciones secuencialmente en dos ciclos de reloj diferentes.

4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

• Algoritmo de la raíz entera

```
Input X
Resul = 0
Temp = 1
Resta = X
while (Resta - Temp) ≥ 0
{  Resta = Resta - Temp
  Resul = Resul + 1
  Temp = Temp + 2 }
output Resul
```

▪ Diseño de la Ruta de Datos.

- Se parte del algoritmo general y, si se puede, se optimiza para implementarlo en hardware.
 - ❑ En este caso las restas se pueden convertir en sumas mediante el C2.
 - ✓ Esto facilita la comprobación del bucle, **(Resta - Temp) ≥ 0** ?
 - El bucle se termina si el resultado de la comprobación es negativo.
 - Mediante la aritmética en C2 se comprueba fácilmente, ya que sólo hay que tener en cuenta el acarreo de salida (Cout).
 - **Se termina el bucle si Cout = 0.**
 - ❑ Restar 1 es lo mismo que sumar -1 en C2 que es un número binario con todos los bits a 1.
 - ❑ Por tanto, al actualizar Temp en vez de sumarle 2 se **le suma -2 en C2 que es 1.... 10.**

• Algoritmo de la raíz entera optimizado.

```
Input X
Resul = 0
Temp = -1C2
Resta = X
while (Resta + Temp) ≥ 0
{  Resta = Resta + Temp
  Resul = Resul + 1
  Temp = Temp + -2C2 }
output Resul
```

4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

❖ Diseño de la Ruta de Datos.

- **Algoritmo de la raíz entera optimizado.**

Input X

Resul = 0

Temp = -1_{C2}

Resta = X

while (Resta + Temp) $\geq$ 0

{ Resta = Resta + Temp

Resul = Resul + 1

Temp = Temp + -2_{C2} }

output Resul

- A partir del algoritmo optimizado se **determinan las microoperaciones.**

Resul $\leftarrow$ 0

Resul $\leftarrow$ Resul + 1

Resta $\leftarrow$ X

Resta $\leftarrow$ Resta + Temp

Temp $\leftarrow$ -1_{C2}

Temp $\leftarrow$ Temp + -2_{C2}

- A partir de las microoperaciones se determinan:
 - Los **registros** y las **operaciones** que deben realizar.
 - Las **unidades funcionales** necesarias.
 - La conexión de todos los componentes.

4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

❖ Diseño de la Ruta de Datos. Componentes (1).

• Microoperaciones:

- $\text{Resul} \leftarrow 0$
- $\text{Resul} \leftarrow \text{Resul} + 1$

▪ Registro Resul.

✓ Dos operaciones:

- Puesta a cero síncrona.
- Incremento de su contenido.

✓ Tipo de registro.

- Contador binario ascendente con clear.

✓ Terminales de control:

- Clear
- Inc

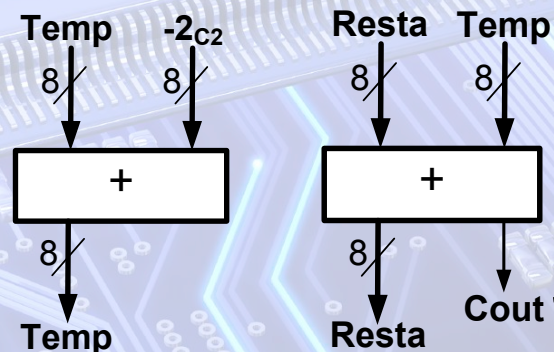
• Supóngase $n = 8$



• Microoperaciones:

- $\text{Resta} \leftarrow \text{Resta} + \text{Temp}$
- $\text{Temp} \leftarrow \text{Temp} + -2_{C2}$

▪ Dos Sumadores



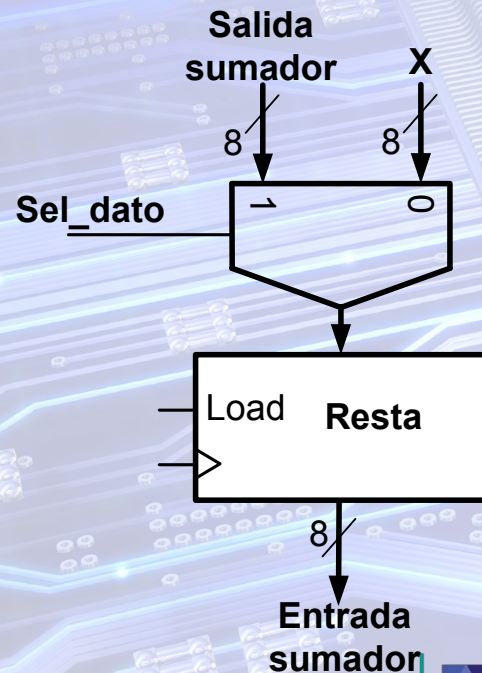
▪ Determina el fin del bucle del algoritmo.

4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

❖ Diseño de la Ruta de Datos. Componentes (2).

• Microoperaciones:

- $\text{Resta} \leftarrow X$
- $\text{Resta} \leftarrow \text{Resta} + \text{Temp}$



▪ Registro Resta.

✓ Operación:

- Carga paralela con 2 fuentes:
 - X
 - $\text{Resta} + \text{Temp}$. Salida del sumador.
- Se necesita un mux de 2 a 1 para seleccionar la fuente de datos a cargar.

✓ Tipo de registro.

- Registro con carga paralela.

✓ Terminales de control:

- Load:
 - Habilita la carga paralela.
- Sel_dato:
 - Selecciona la fuente de datos que se carga en el registro.

❖ Sel_dato = 0

⇒ X

❖ Sel_dato = 1

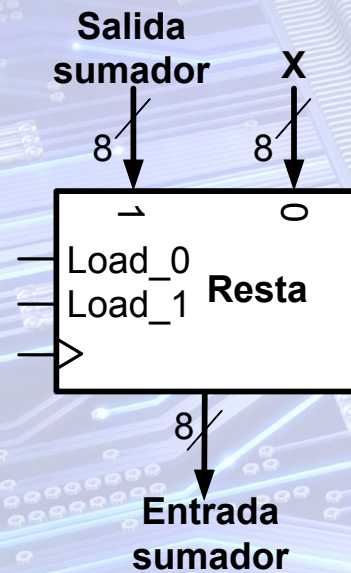
⇒

Salida sumador

4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

• Microoperaciones:

- $\text{Resta} \leftarrow X$
- $\text{Resta} \leftarrow \text{Resta} + \text{Temp}$



■ Diseño de la Ruta de Datos. Componentes (2).

✓ Registro Resta modificado.

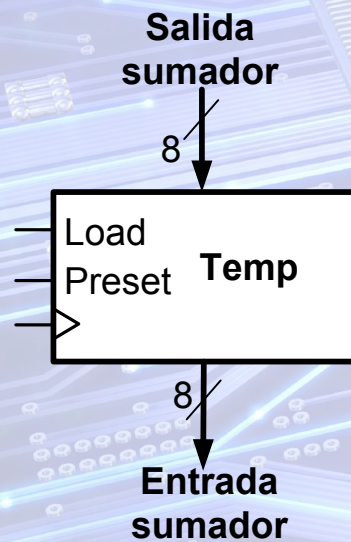
- Para simplificar la descripción VHDL se incluye el mux de 2 a 1 en el registro.
- **Terminales de control:**
 - ☐ **Load_0:**
 - Habilita la carga paralela con la entrada de datos 0.
 - Carga X.
 - ☐ **Load_1:**
 - Habilita la carga paralela con la entrada de datos 1.
 - Carga Resta + Temp.

4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

■ Diseño de la Ruta de Datos. Componentes (3).

• Microoperaciones:

- $\text{Temp} \leftarrow -1_{C2}$
- $\text{Temp} \leftarrow \text{Temp} + -2_{C2}$



• Registro Temp.

✓ Operación:

- ☐ Puesta a 1 síncrona. Preset.
- ☐ Carga paralela con la salida del sumador.

✓ Tipo de registro.

- ☐ Registro con carga paralela y puesta a 1 síncrona.

✓ Terminales de control:

- ☐ Preset:
 - Puesta a 1 síncrona.
- ☐ Load:
 - Habilita la carga paralela.

4. Descripción FSM+D en VHDL.

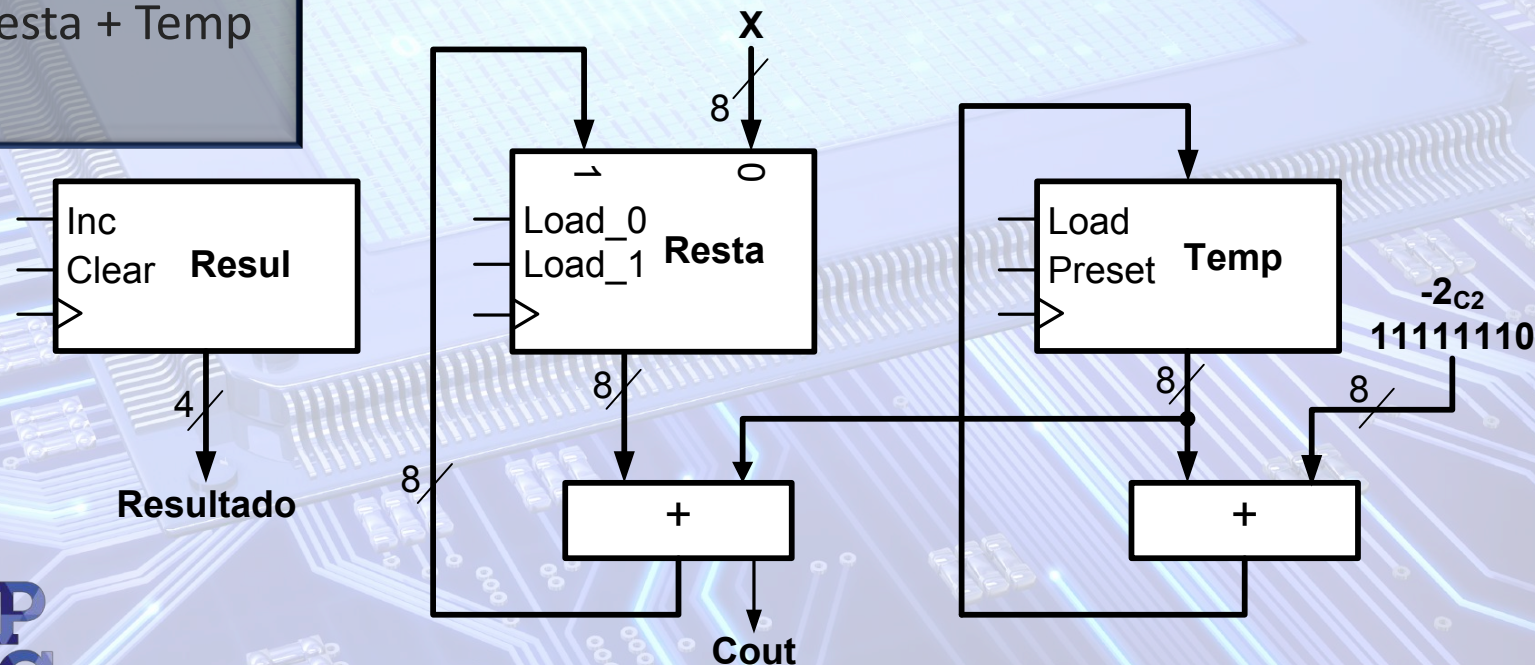
Diseño de la Unidad de Cálculo de la Raíz Cuadrada

• Microoperaciones:

- $\text{Resul} \leftarrow 0$
- $\text{Resul} \leftarrow \text{Resul} + 1$
- $\text{Temp} \leftarrow -1_{C2}$
- $\text{Temp} \leftarrow \text{Temp} + -2_{C2}$
- $\text{Resta} \leftarrow X$
- $\text{Resta} \leftarrow \text{Resta} + \text{Temp}$

❖ Diseño de la Ruta de Datos.

- Se conectan todos los componentes para realizar las microoperaciones.



4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

❖ Diseño de la Ruta de Control.

- Algoritmo de la raíz entera optimizado.

Input X

Resul = 0

Temp = -1_{C2}

Resta = X

while (Resta + Temp) ≥ 0

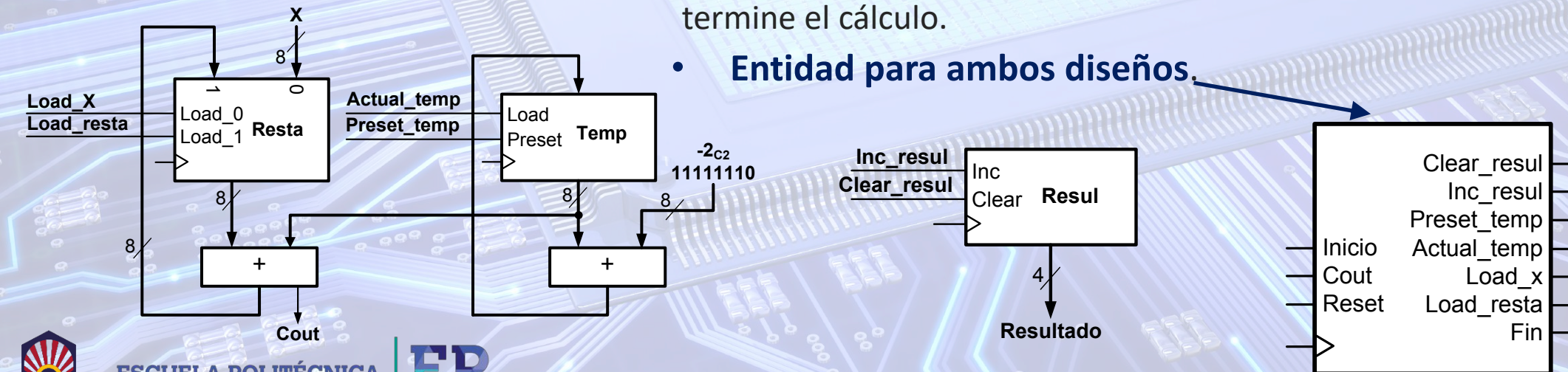
{ Resta = Resta + Temp

Resul = Resul + 1

Temp = Temp + -2_{C2} }

output Resul

- A partir del algoritmo optimizado se diseña la FSM que genera la secuencia de microoperaciones para implementarlo.
 - Debe activar los Terminales de Control, a través de sus salidas de control, en la secuencia y en los instantes adecuados.
- Se puede diseñar por Mealy o por Moore.
- La entrada de control **Inicio** empieza el cálculo.
- Generará una salida de control **Fin**, que se activará cuando se termine el cálculo.
- Entidad para ambos diseños.



4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

- Algoritmo de la raíz entera optimizado.

Input X

Resul = 0

Temp = -1_{C2}

Resta = X

while (Resta + Temp) ≥ 0

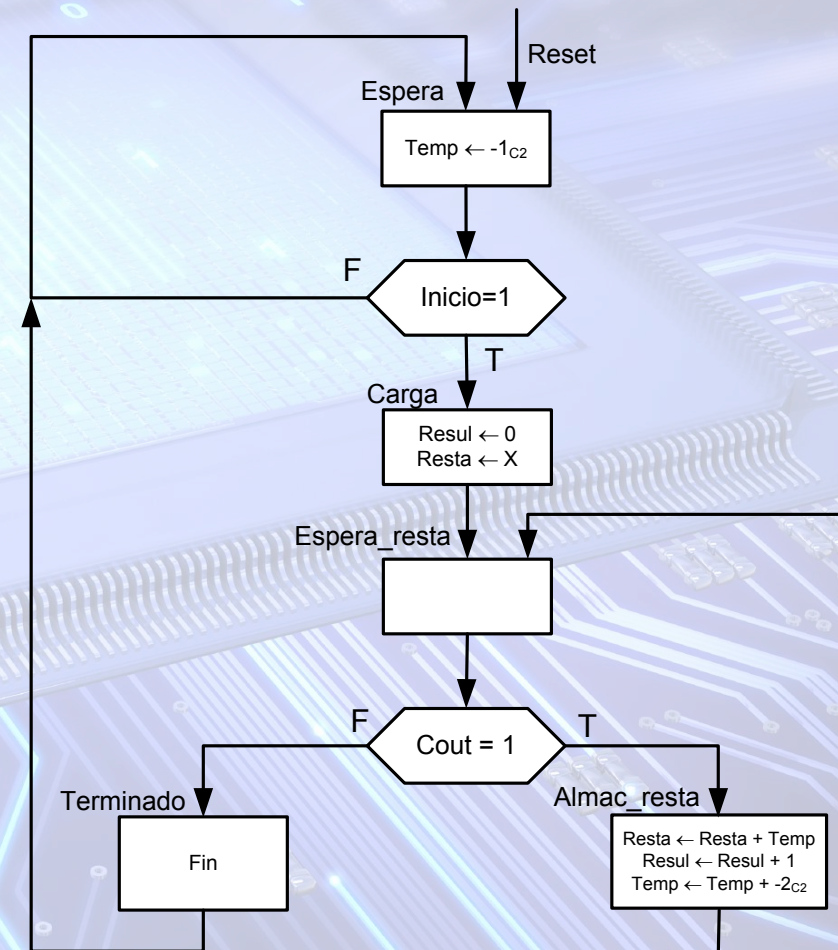
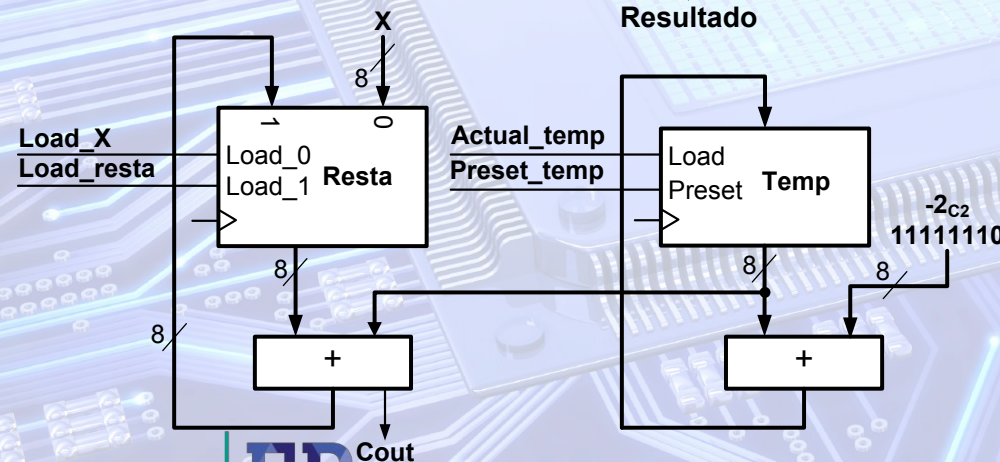
{ Resta = Resta + Temp

Resul = Resul + 1

Temp = Temp + -2_{C2} }

output Resul

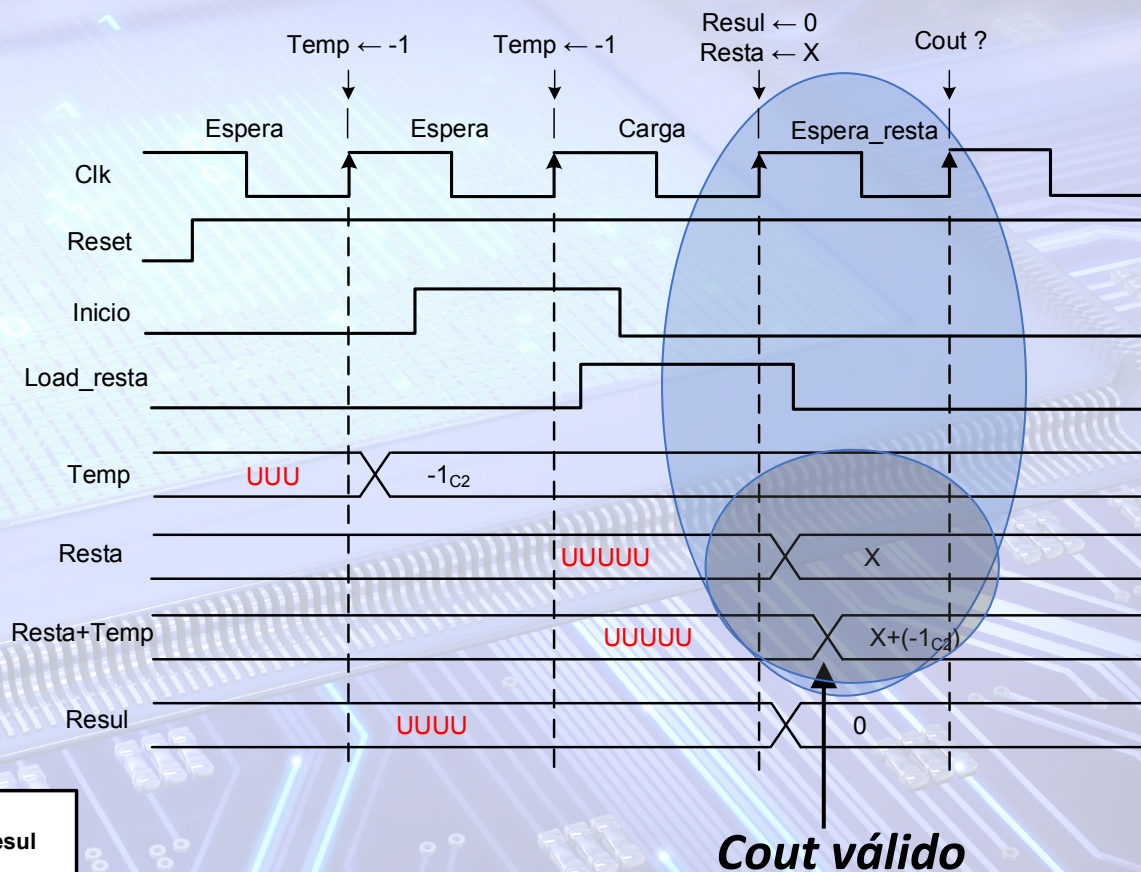
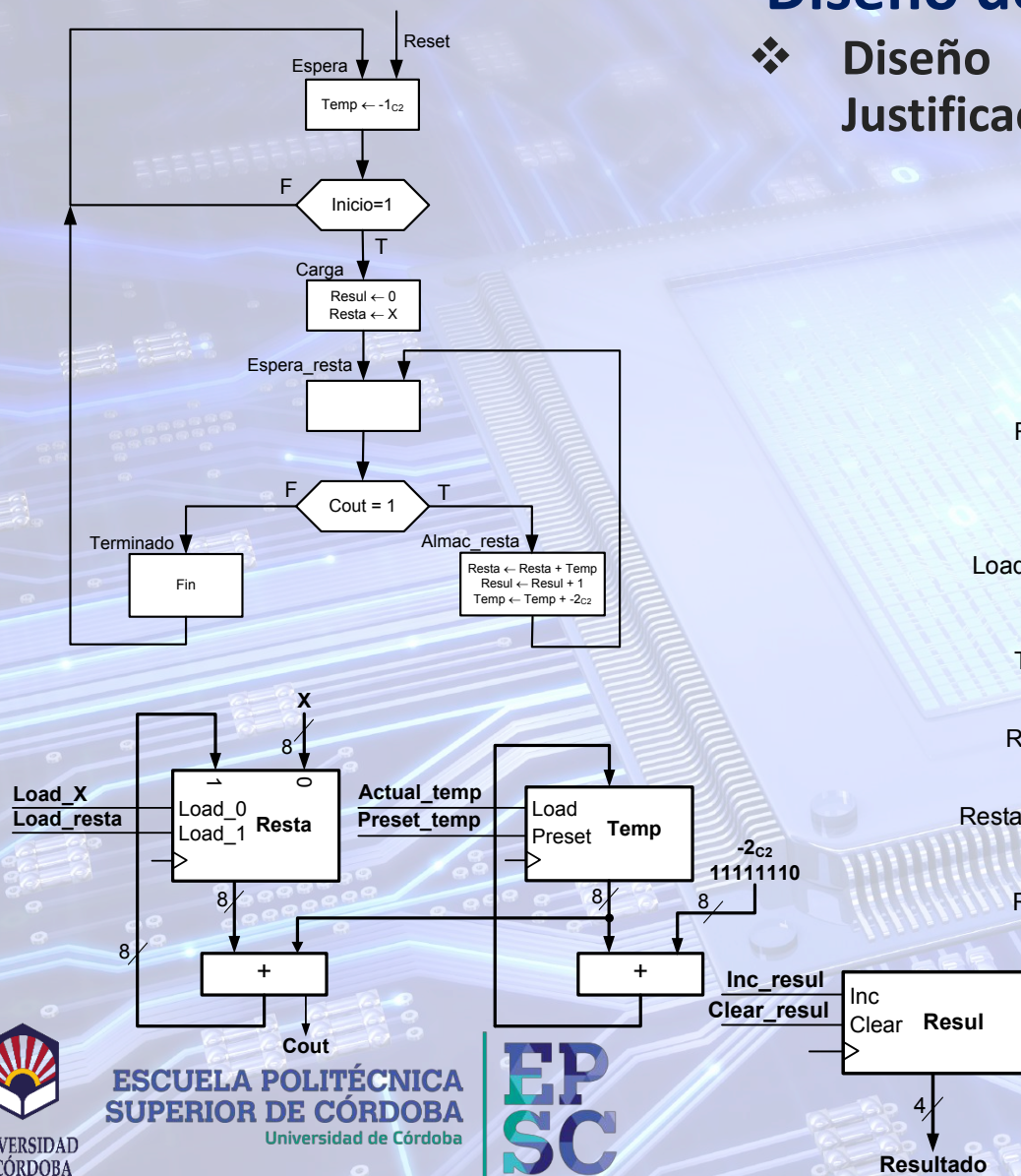
- ❖ Diseño de la Ruta de Control. Diseño por Moore.
 - Diagrama ASM. Secuencia de microoperaciones.



4. Descripción FSM+D en VHDL.

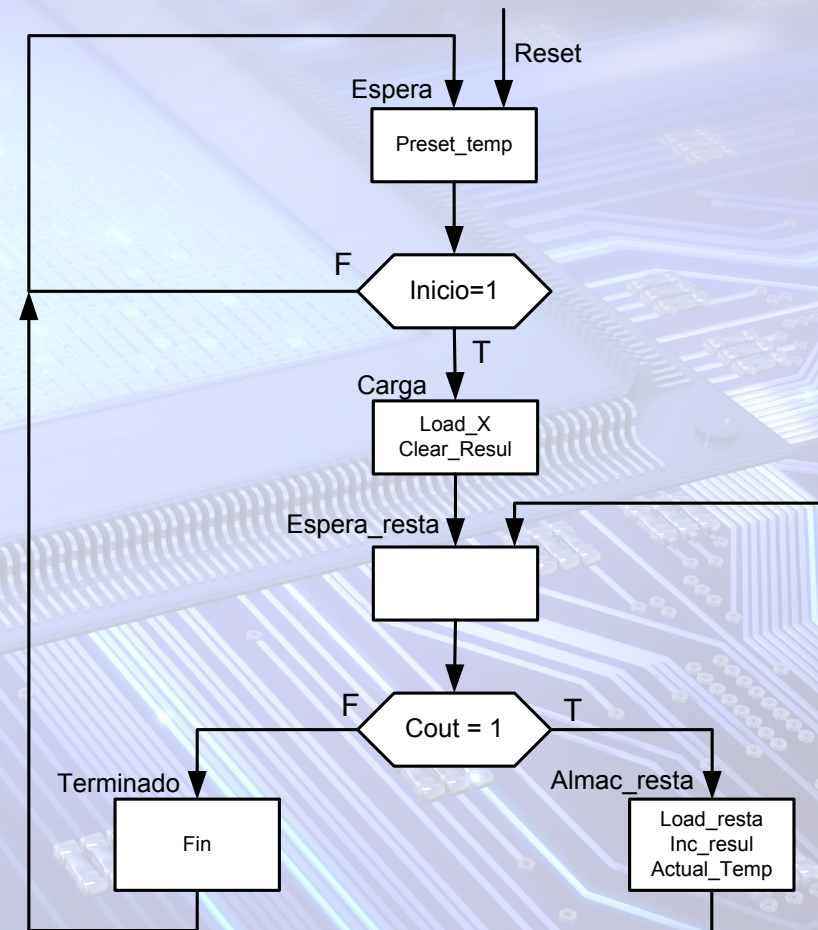
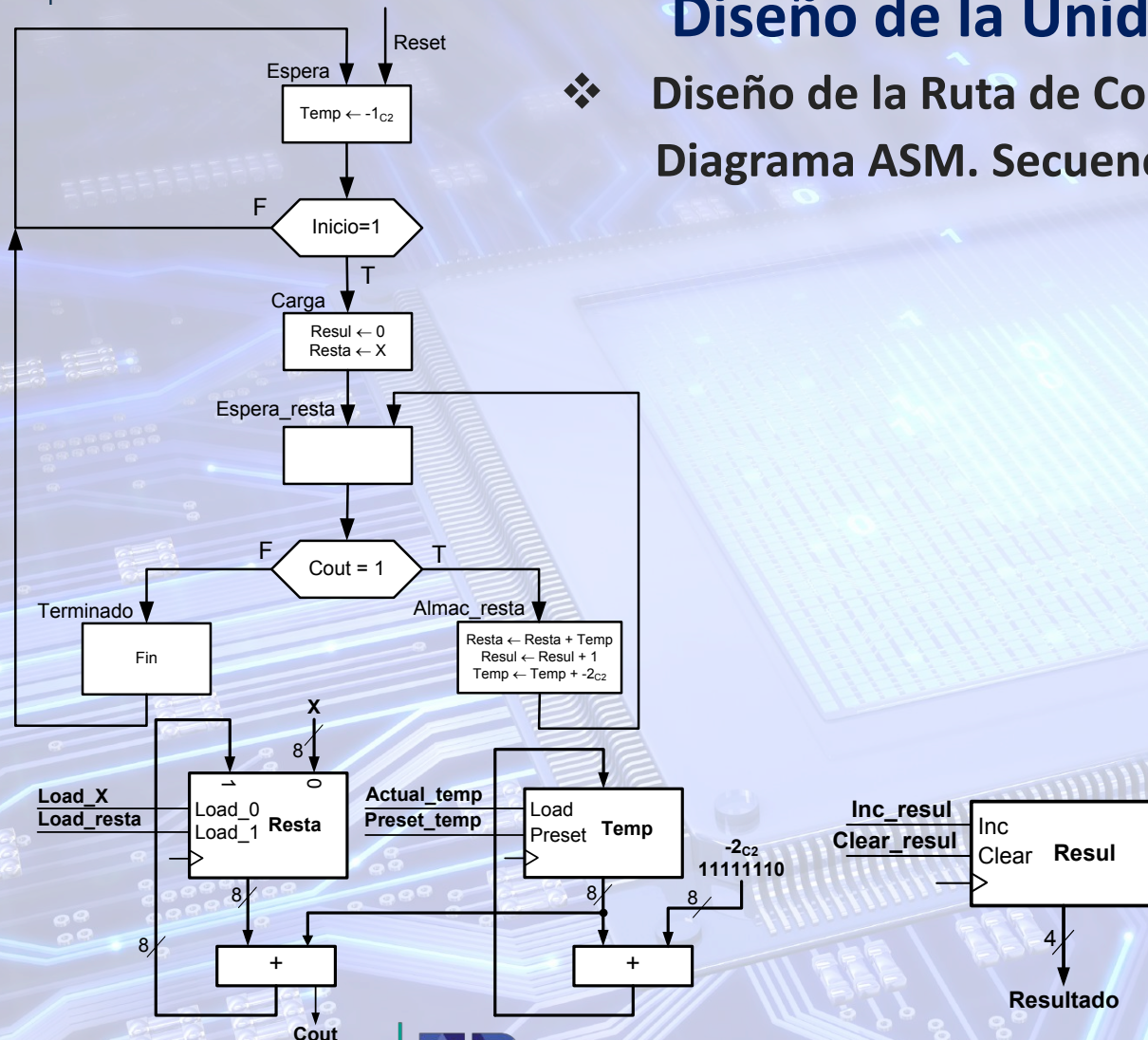
Diseño de la Unidad de Cálculo de la Raíz Cuadrada

- ❖ Diseño de la Ruta de Control. Diseño por Moore. Justificación del estado Espera_resta.



4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

- ❖ **Diseño de la Ruta de Control. Diseño por Moore.**
Diagrama ASM. Secuencia de las señales de control.



4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

- ❖ Diseño de la Ruta de Control. Diseño por Mealy.
Diagrama ASM. Secuencia de microoperaciones.

- Algoritmo de la raíz entera optimizado.

Input X

Resul = 0

Temp = -1_{C2}

Resta = X

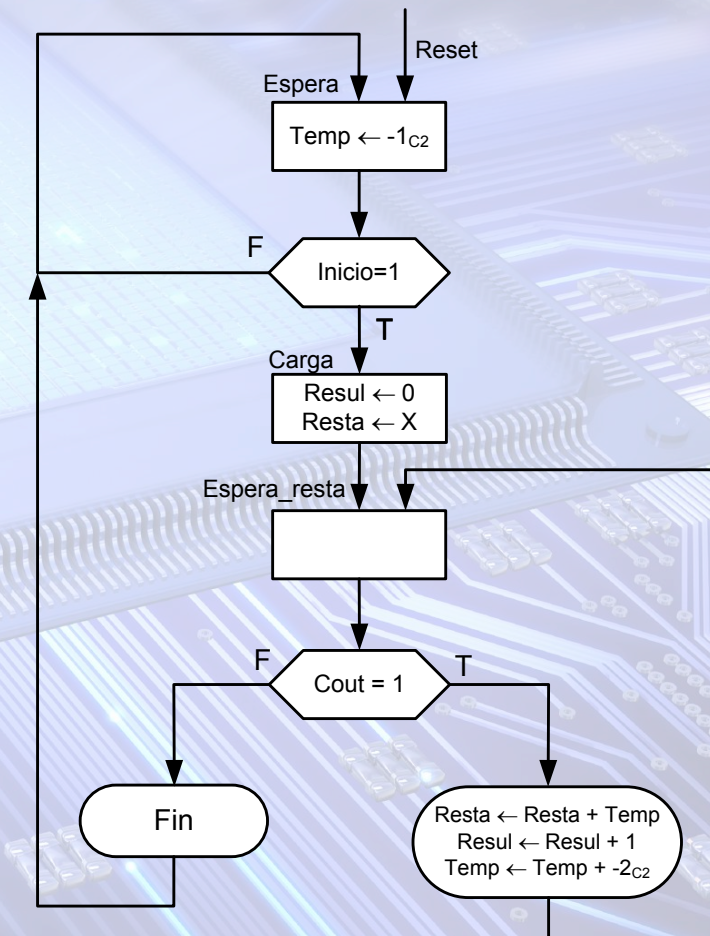
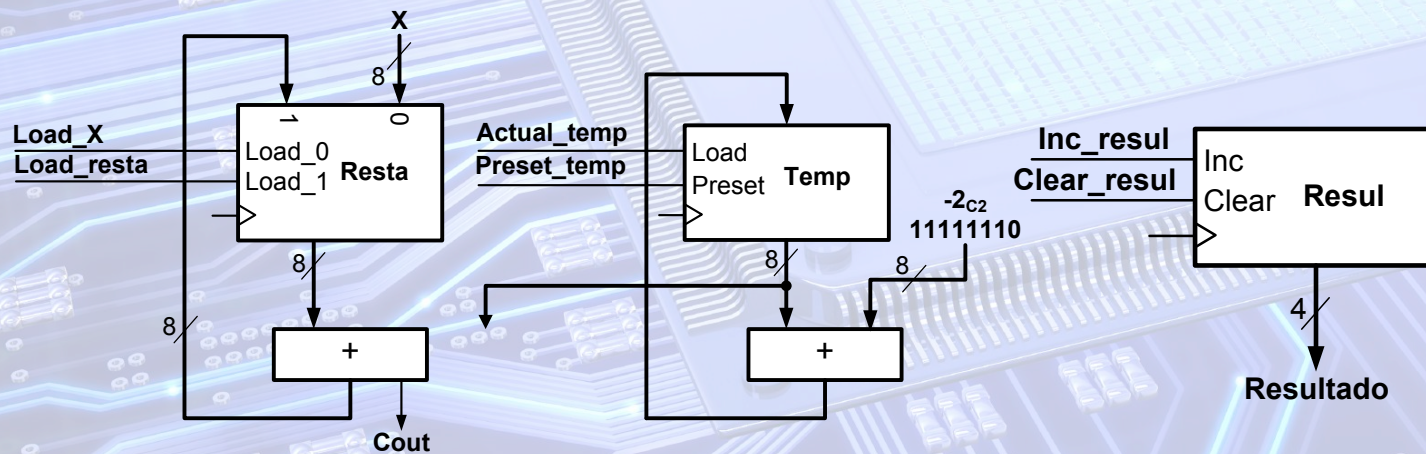
while (Resta + Temp) ≥ 0

{ Resta = Resta + Temp

Resul = Resul + 1

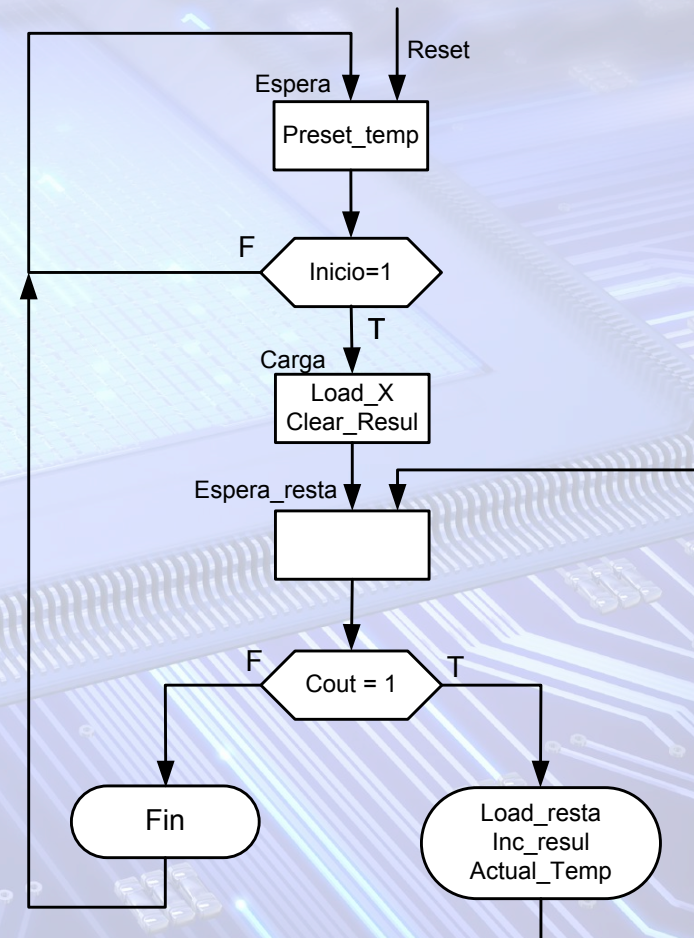
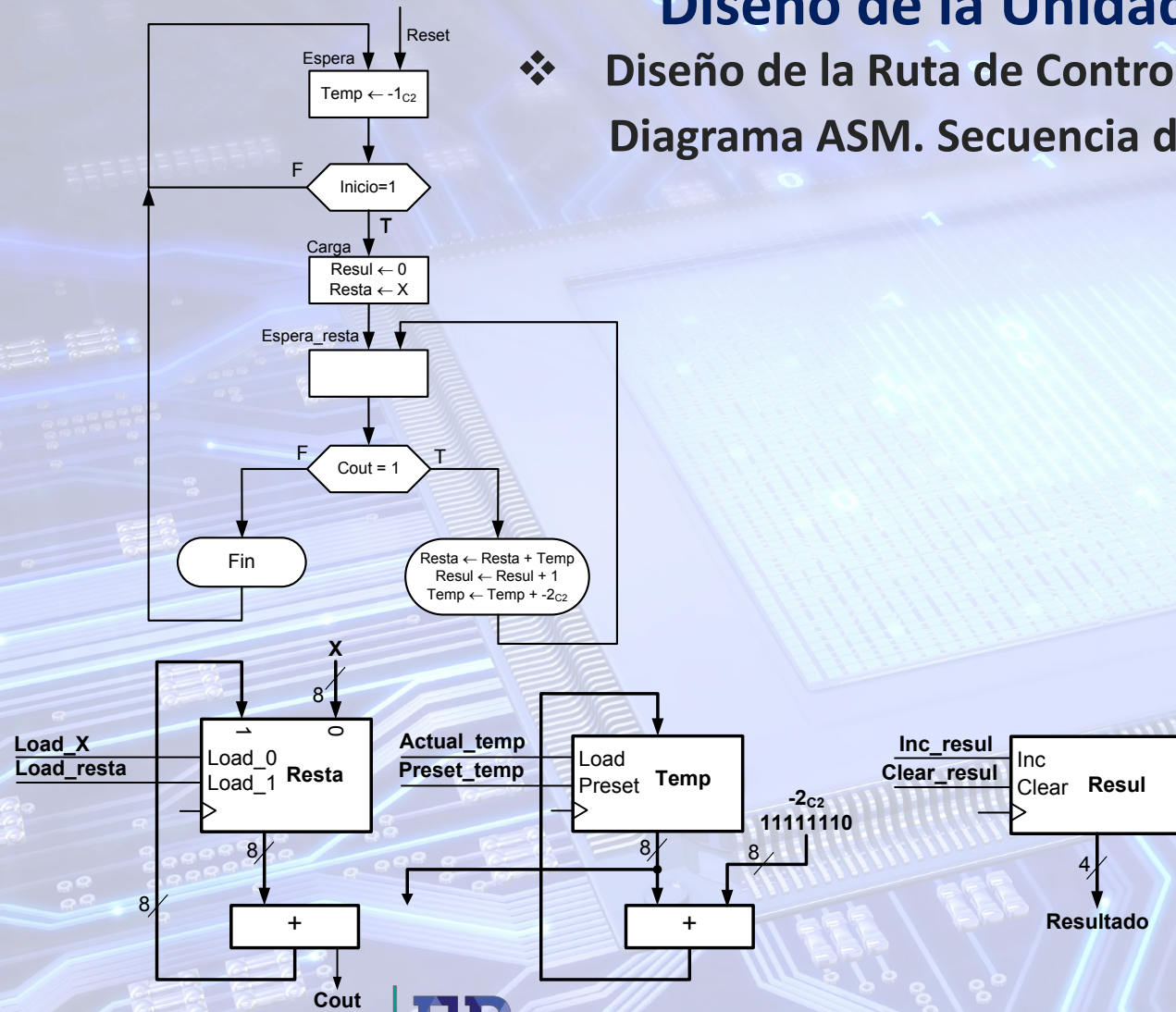
Temp = Temp + -2_{C2} }

output Resul



4. Descripción FSM+D en VHDL. Diseño de la Unidad de Cálculo de la Raíz Cuadrada

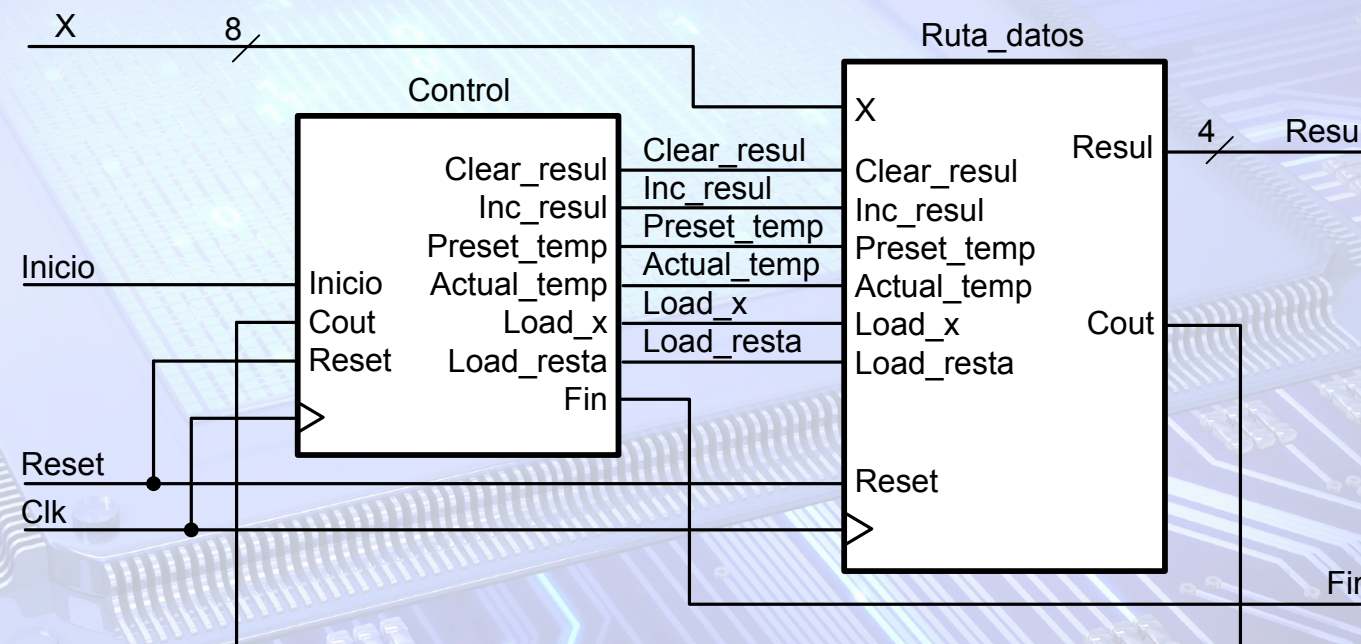
❖ Diseño de la Ruta de Control. Diseño por Mealy.
Diagrama ASM. Secuencia de las señales de control.



4. Descripción FSM+D en VHDL.

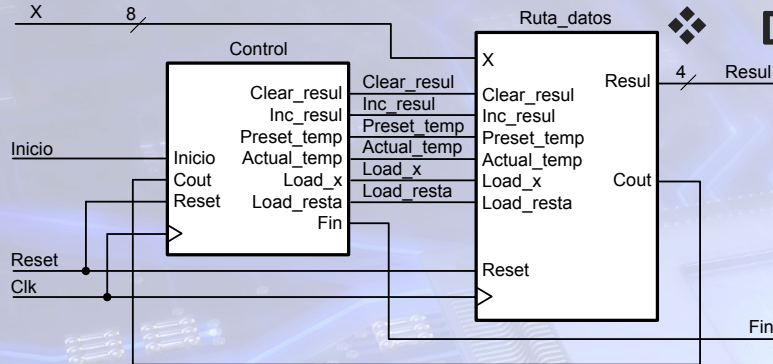
Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

- ❖ La descripción se hace como un diseño jerárquico.
Estructura del nivel raíz.



4. Descripción FSM+D en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada



Descripción del nivel raíz

architecture Behavioral of Raiz_cuad_fsm_m_d is

component Ruta_datos is Port (

```

    Clk : in std_logic;
    reset : in std_logic;
    X : in std_logic_vector(7 downto 0);
    Clear_resul : in std_logic;
    Inc_resul : in std_logic;
    preset_temp : in std_logic;
    Actual_temp : in std_logic;
    Load_x : in std_logic;
    Load_resta : in std_logic;
    Cout : out std_logic;
    Result : out std_logic_vector(3 downto 0);
  
```

end component Ruta_datos;

component Control is Port (

```

    Clk : in std_logic;
    reset : in std_logic;
    Inicio : in std_logic;
    Cout : in std_logic;
    Clear_resul : out std_logic;
    Inc_resul : out std_logic;
    preset_temp : out std_logic;
    Actual_temp : out std_logic;
    Load_x : out std_logic;
    Load_resta : out std_logic;
    Fin : out std_logic;
  
```

end component Control;

```

signal Clear_resul, Inc_resul, preset_temp : std_logic;
signal Cout, Actual_temp, Load_x, Load_resta : std_logic;
  
```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_unsigned.ALL;

entity Raiz_cuad_fsm_m_d is

Port (Clk : in std_logic;

reset : in std_logic;

Inicio : in std_logic;

X : in std_logic_vector(7 downto 0);

Fin : out std_logic;

Result : out std_logic_vector(3 downto 0)

);

end Raiz_cuad_fsm_m_d;

4. Descripción

FSM+D

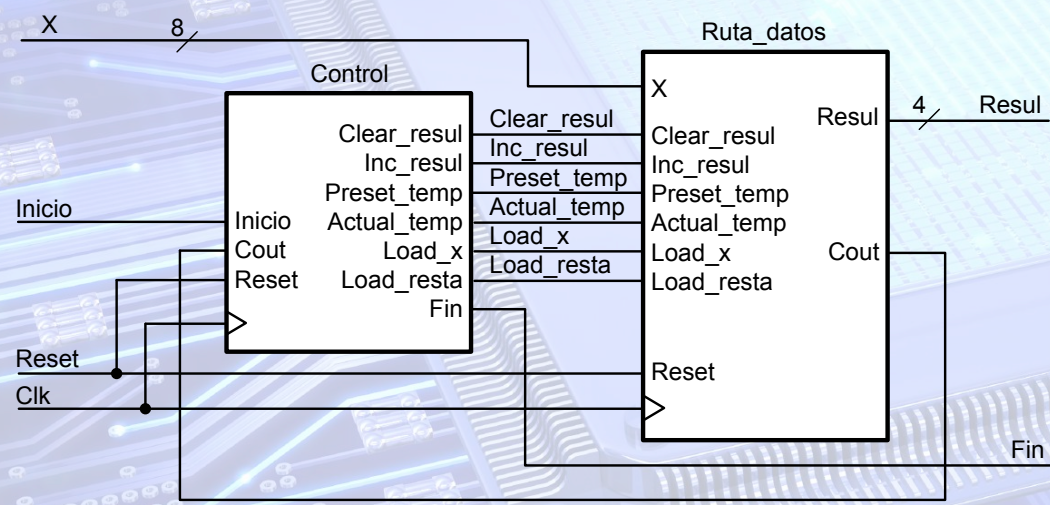
en

VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción del nivel raíz (2).

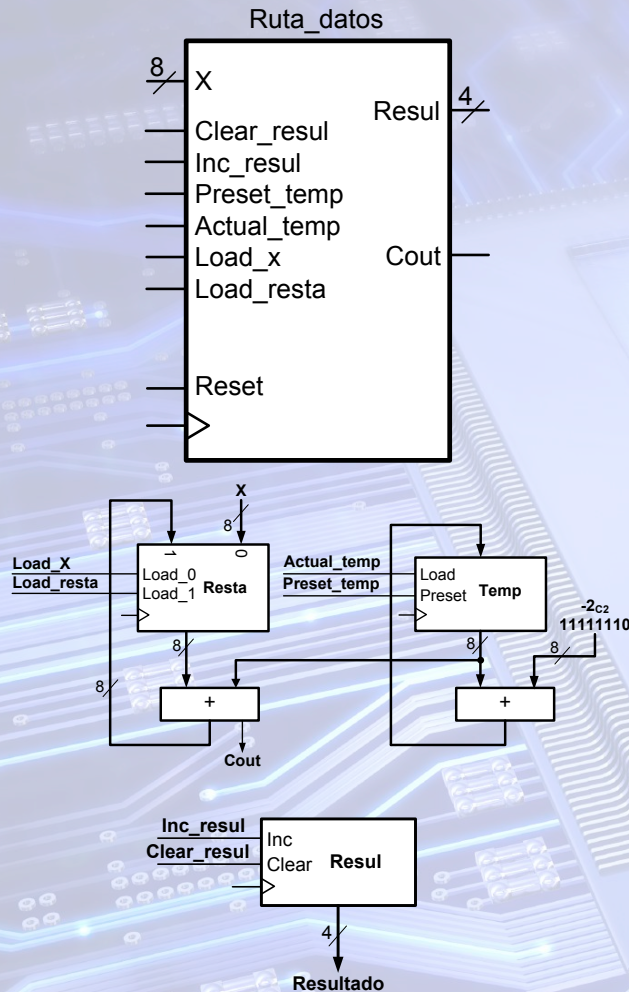
- Se conectan la ruta de datos y la de control mediante las señales de control



```
begin
  U0 : Ruta_datos Port map(
    Clk => Clk,
    Reset => Reset,
    X => X,
    Clear_resul => Clear_resul,
    Inc_resul => Inc_resul,
    Preset_temp => Preset_temp,
    Actual_temp => Actual_temp,
    Load_x => Load_x,
    Load_resta => Load_resta,
    Cout => Cout,
    Result => Result
  );
  U1 : Control Port map(
    Clk => Clk,
    reset => Reset,
    Inicio => Inicio,
    Cout => Cout,
    Clear_resul => Clear_resul,
    Inc_resul => Inc_resul,
    preset_temp => Preset_temp,
    Actual_temp => Actual_temp,
    Load_x => Load_x,
    Load_resta => Load_resta,
    Fin => Fin
  );
end Behavioral;
```

4. Descripción FSM+D en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
entity Ruta_datos is
    Port ( Clk : in std_logic;
    reset : in std_logic;
    X : in std_logic_vector(7 downto 0);
    Clear_resul : in std_logic;
    Inc_resul : in std_logic;
    preset_temp : in std_logic;
    Actual_temp : in std_logic;
    Load_x : in std_logic;
    Load_resta : in std_logic;
    Cout : out std_logic;
    Resul : out std_logic_vector(3 downto 0));
end Ruta_datos;
architecture Behavioral of Ruta_datos is
    signal Resul_temp : std_logic_vector(3 downto 0);
    signal Temp : std_logic_vector (7 downto 0);
    signal Resta : std_logic_vector (7 downto 0);
    signal Suma : std_logic_vector (8 downto 0);
Begin
    process(Clk, reset)
    begin
        If reset = '1' then
            Resul_temp <= (others => '0');
            Temp <= (others => '1');
        Elself Clk'event and Clk = '1' then
```

```
-- Descripcion registro Resultado
        If Clear_resul = '1' then
            Resul_temp <= (others => '0');
        Elself Inc_resul = '1' then
            Resul_temp <= Resul_temp + 1;
        End if;

-- Descripcion registro Temp
        If Preset_temp = '1' then
            Temp <= (others => '1');
        Elself Actual_temp = '1' then
            Temp <= Temp + "11111110";
        End if;

-- Descripcion registro Resta
        If Load_X = '1' then
            resta <= X;
        Elself Load_resta = '1' then
            Resta <= Suma(7 downto 0);
        End if;
    End if;
End process;

Suma <= ('0' & Resta) + ('0' & temp);
Resul <= Resul_temp;
Cout <= Suma(8);

End Behavioral;
```

4.

Descripción

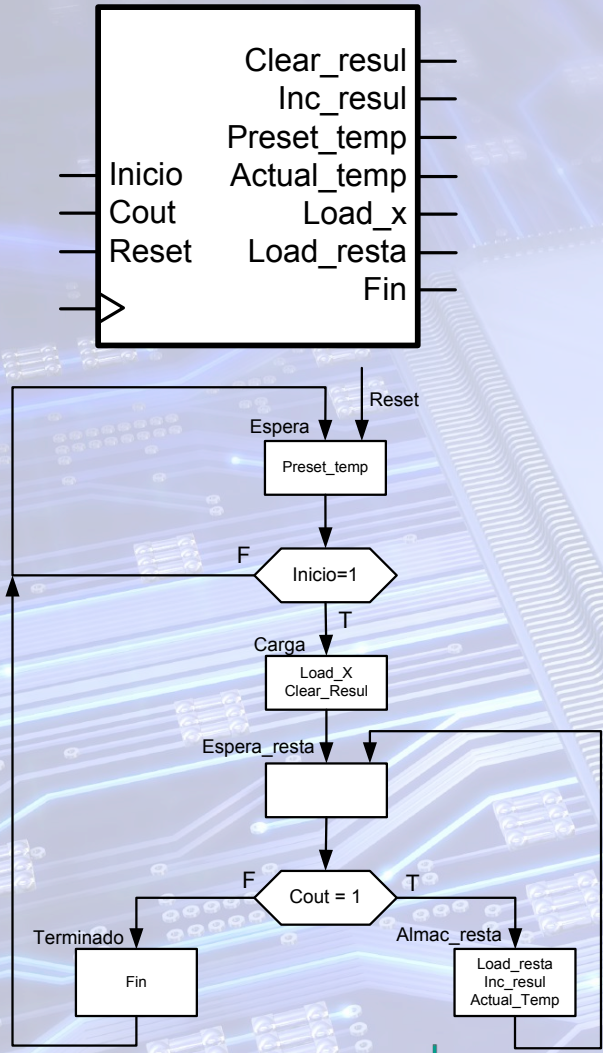
FSM+D

en

VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción de la ruta de control (Moore) (1).



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity Control is
    Port ( Clk : in std_logic;
          reset : in std_logic;
          Inicio : in std_logic;
          Cout : in std_logic;
          Clear_resul : out std_logic;
          Inc_resul : out std_logic;
          preset_temp : out std_logic;
          Actual_temp : out std_logic;
          Load_x : out std_logic;
          Load_resta : out std_logic;
          Fin : out std_logic;
          Resul : out std_logic_vector(3 downto 0));
end Control;
```

```
architecture Behavioral of Control is
    type Estados is (Espera, Carga, Espera_resta,
                    Almac_Resta, Terminado);
    signal Estado_actual, Estado_siguiente : estados;
begin

    process(Clk, reset)
    begin
        If reset = '1' then
            Estado_actual <= Espera;
        Elself Clk'event and Clk = '1' then
            Estado_actual <= Estado_siguiente;
        End if;
    End process;
end Behavioral;
```

4.

Descripción

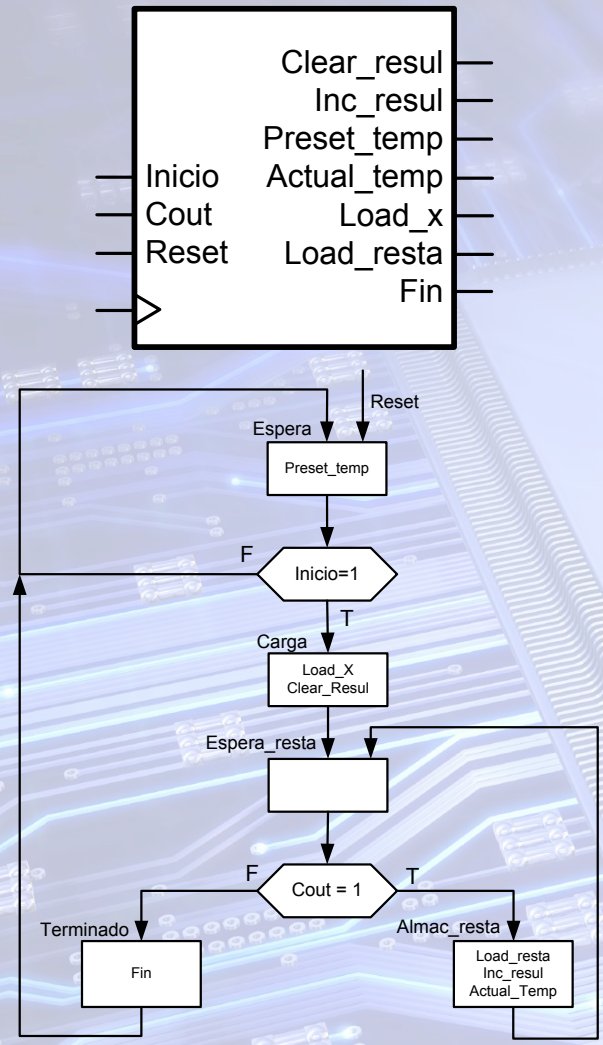
FSM+D

en

VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción de la ruta de control (Moore) (2).



```
process(Estado_actual, Inicio, Cout)
begin

    Preset_temp <= '0';
    Load_X <= '0';
    Clear_Resul <= '0';
    Load_resta <= '0';
    Inc_Resul <= '0';
    Actual_temp <= '0';
    Fin <= '0';

    case Estado_actual is
        when Espera =>
            Preset_Temp <= '1';
            If Inicio = '1' Then
                Estado_siguiente <= Carga;
            else
                Estado_siguiente <= Espera;
            End if;
        when Carga =>
            Load_X <= '1';
            Clear_Resul <= '1';
            Estado_siguiente <= Espera_resta;
```

```
        when Espera_resta =>
            If Cout = '1' then
                Estado_siguiente <= Almac_Resta;
            else
                estado_siguiente <= Terminado;
            end if;
        when Almac_Resta =>
            Load_Resta <= '1';
            Inc_Resul <= '1';
            Actual_Temp <= '1';
            Estado_siguiente <= Espera_resta;
        when Terminado =>
            Fin <= '1';
            Estado_siguiente <= Espera;

        when others =>
            Estado_siguiente <= Espera;
    end case;
end process;

end Behavioral;
```

4.

Descripción

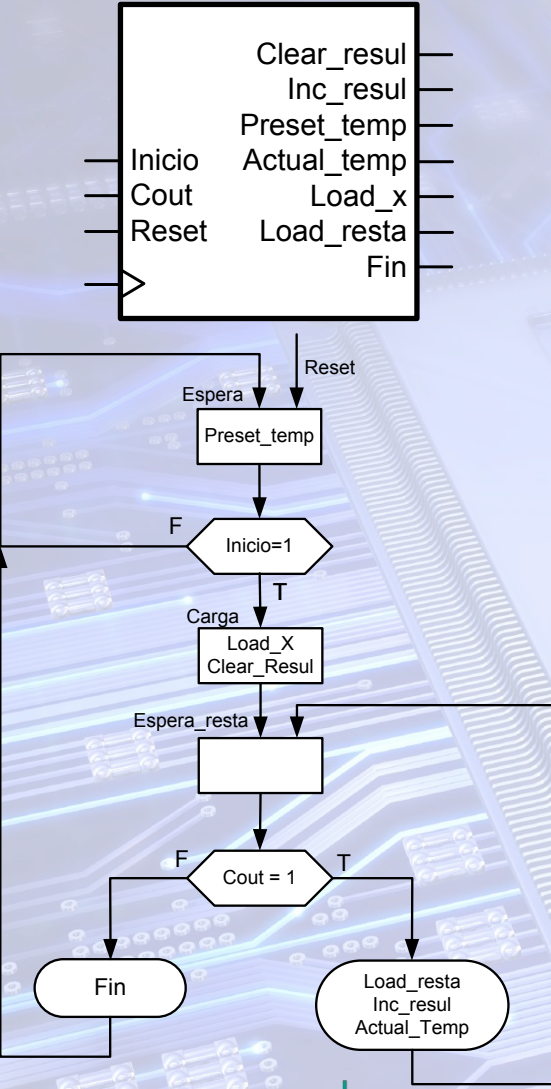
FSM+D

en

VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción de la ruta de control (Mealy) (1).



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity Control is
Port ( Clk : in std_logic;
reset : in std_logic;
Inicio : in std_logic;
Cout : in std_logic;
Clear_resul : out std_logic;
Inc_resul : out std_logic;
Actual_temp : out std_logic;
Load_x : out std_logic;
Load_resta : out std_logic;
Fin : out std_logic;
Resul : out std_logic_vector(3 downto 0));
end Control;
```

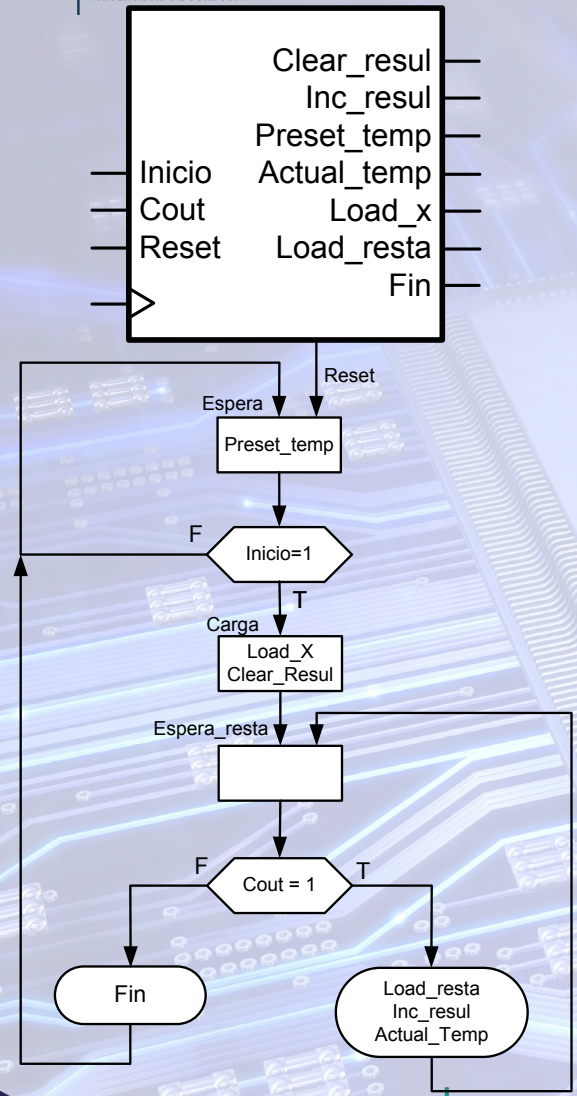
```
architecture Behavioral of Control is
type Estados is (Espera, Carga, Espera_resta);
signal Estado_actual, Estado_siguiente : Estados;
begin

process(Clk, reset)
begin
    If reset = '1' then
        Estado_actual <= Espera;
    Elsif Clk'event and Clk = '1' then
        Estado_actual <= Estado_siguiente;
    End if;
End process;
```

4. Descripción FSM+D en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción de la ruta de control (Mealy) (2).



```
process(Estado_actual, Inicio, Cout)
begin

    Preset_temp <= '0';
    Load_X <= '0';
    Clear_Resul <= '0';
    Load_resta <= '0';
    Inc_Resul <= '0';
    Actual_temp <= '0';
    Fin <= '0';

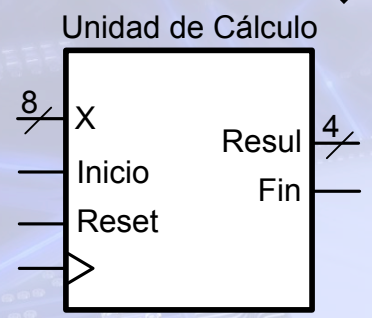
    case Estado_actual is
        when Espera =>
            Preset_Temp <= '1';
            If Inicio = '1' then
                Estado_siguiente <= Carga;
            else
                Estado_siguiente <= Espera;
            End if;
    end case;
```

```
when Carga =>
    Load_X <= '1';
    Clear_Resul <= '1';
    Estado_siguiente <= Espera_resta;
when Espera_resta =>
    If Cout = '1' then
        Estado_siguiente <= Espera_Resta;
        Load_Resta <= '1';
        Inc_Resul <= '1';
        Actual_Temp <= '1';
    else
        estado_siguiente <= Espera;
        Fin <= '1';
    end if;
when others =>
    Estado_siguiente <= Espera;
end case;
end process;

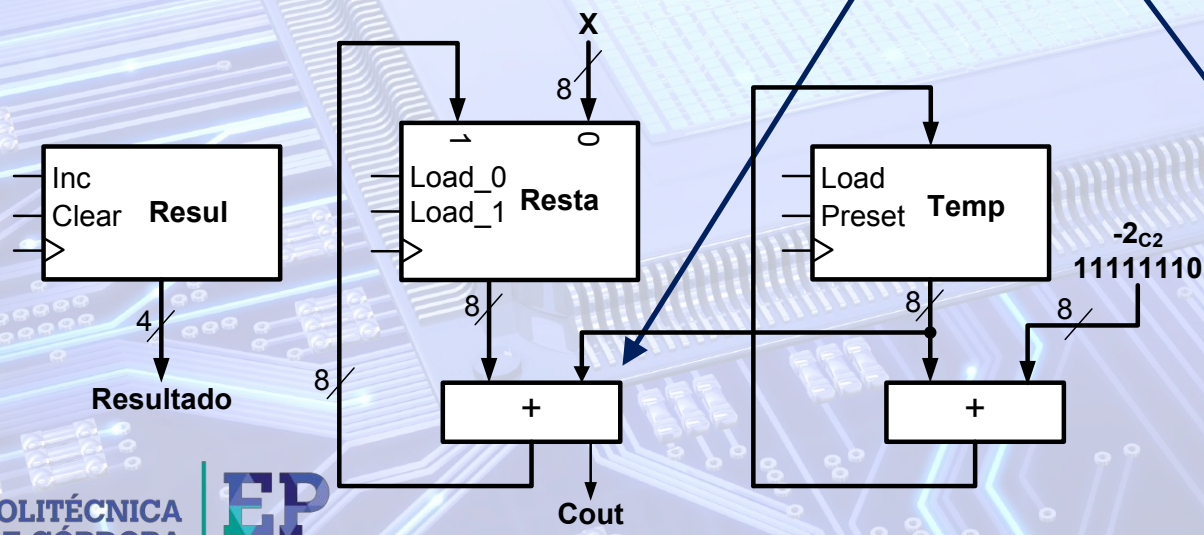
end Behavioral;
```

4. Descripción FSMD en VHDL. Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción como FSMD Moore (1).



- Entidad.
- Arquitectura.
 - Se declaran las señales que implementan los registros y el sumador.



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity Raiz_cuad_fsmd is
Port ( Clk : in std_logic;
reset : in std_logic;
Inicio : in std_logic;
X : in std_logic_vector(7 downto 0);
Fin : out std_logic;
Resul : out std_logic_vector(3 downto 0));
end Raiz_cuad_fsmd;

architecture Behavioral of Raiz_cuad_fsmd is
type Estados is (Espera, Carga, Espera_resta, Almac_Resta, Terminado);
signal Estado_actual : estados;
signal Resul_temp : std_logic_vector(3 downto 0);
signal Temp : std_logic_vector(7 downto 0);
signal Resta : std_logic_vector(7 downto 0);
signal Suma : std_logic_vector(8 downto 0);
begin
```

- Sólo se declara la señal Estado_actual.

4. Descripción FSMD en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción como FSMD Moore (2).

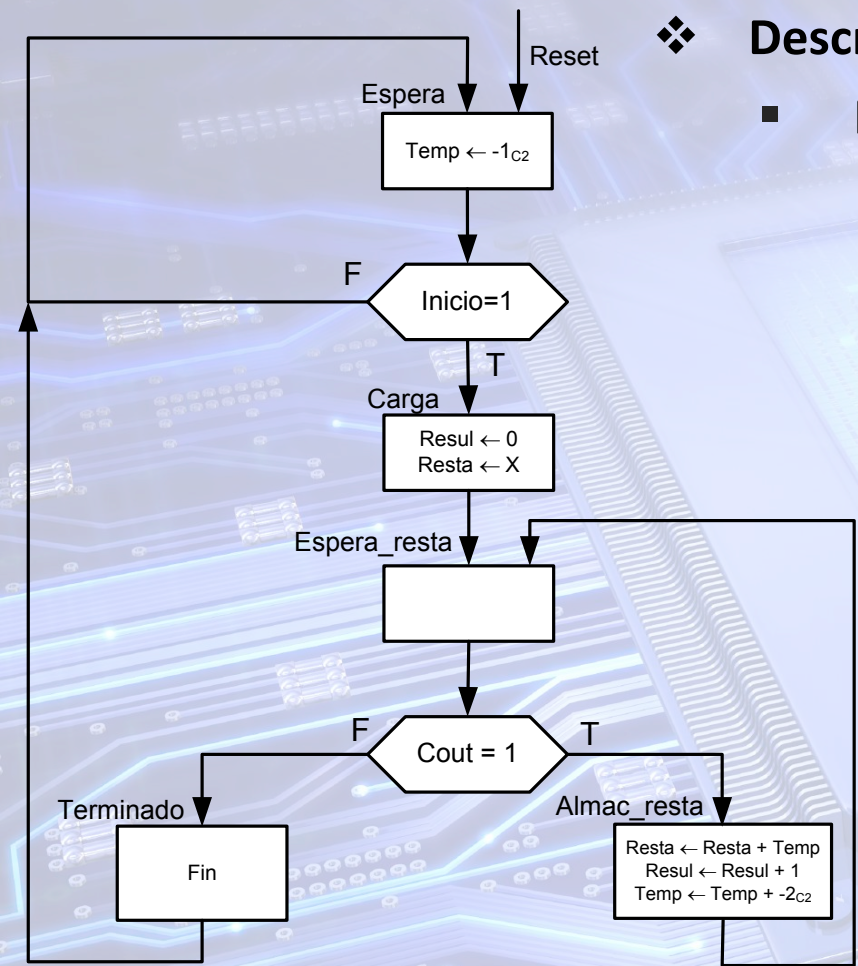
▪ Process del registro de estado

- Se describe la secuencia de estados de la FSM.
- En cada estado de la FSM se indica la microoperación que debe realizarse, y por tanto, el funcionamiento de los registros. $Temp \leftarrow -1_{C2}$

- #### ▪ Microoperaciones:
- $Resta \leftarrow X$
 - $Resul \leftarrow 0$

- Por tanto, la FSM describe la Ruta de Control y la de Datos.
- Por ello, se denomina FSMD.

```
process(Clk, reset)
begin
  If reset = '1' then
    Estado_actual <= Espera;
    Resul_temp <= (others => '0');
    Temp <= (others => '1');
  Elsif Clk'event and Clk = '1' then
    case Estado_actual is
      when Espera =>
        Temp <= (others => '1');
        If Inicio = '1' then
          Estado_actual <= Carga;
        End if;
      when Carga =>
        Resta <= X;
        Resul_temp <= (others => '0');
        Estado_actual <= Espera_resta;
      when Espera_resta =>
        If Suma(8) = '1' then
          Estado_actual <= Almac_Resta;
        else
          Estado_actual <= Terminado;
        end if;
    end case;
  end if;
```



4. Descripción FSMD en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

❖ Descripción como FSMD Moore (3).

▪ Process del registro de estado

▪ Microoperaciones:

- $Resta \leftarrow Resta + Temp$
- $Resul \leftarrow Resul + 1$
- $Temp \leftarrow Temp + -2_{C2}$

```

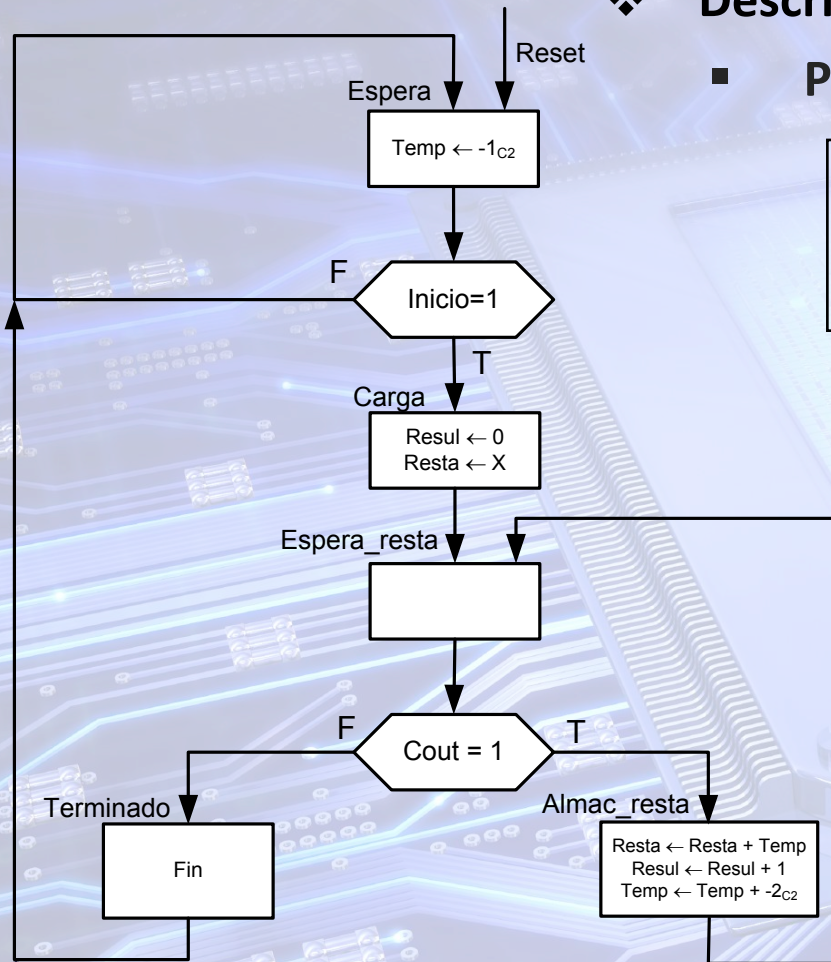
when Almac_Resta =>
    Resta <= Suma(7 downto 0);
    Resul_temp <= resul_temp + 1;
    Temp <= Temp + "11111110";
    Estado_actual <= Espera_resta;
when Terminado =>
    Estado_actual <= Espera;
when others =>
    Estado_actual <= Espera;
end case;
end if;
end process;

Suma <= ('0' & Resta) + ('0' & temp);
Resul <= Resul_temp;

Fin <= '1' when estado_actual = Terminado
      else '0';

end Behavioral;
  
```

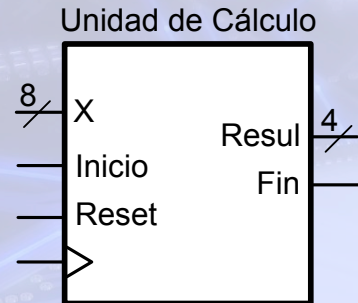
- En un process combinacional y/o con statements concurrentes de asignación de valores a señales se describen el resto de componentes combinacionales (sumador) y las salidas de control (Fin).



4. Descripción FSMD en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

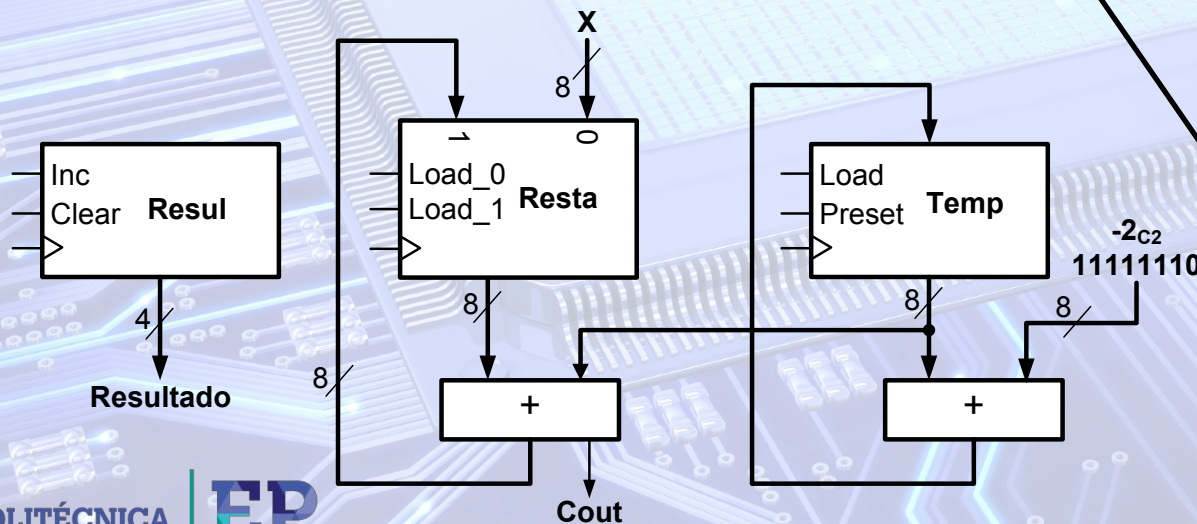
Descripción como FSMD Mealy (1).



Entidad.

Arquitectura.

- Se declaran las señales que implementan los registros y el sumador.



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity Raiz_cuad_fsmd is
  Port ( Clk : in std_logic;
        reset : in std_logic;
        Inicio : in std_logic;
        X : in std_logic_vector(7 downto 0);
        Fin : out std_logic;
        Resul : out std_logic_vector(3 downto 0));
end Raiz_cuad_fsmd;

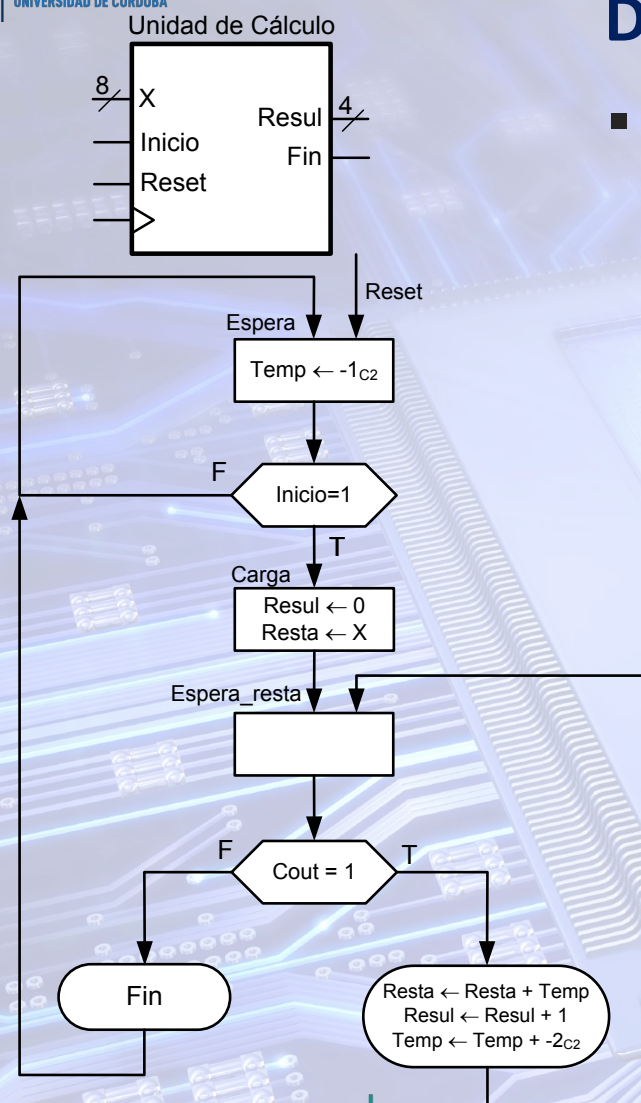
architecture Behavioral of Raiz_cuad_fsmd is
  type Estados is (Espera, Carga, Espera_resta);
  signal Estado_actual : estados;
  signal Resul_temp : std_logic_vector(3 downto 0);
  signal Temp : std_logic_vector(7 downto 0);
  signal Resta : std_logic_vector(7 downto 0);
  signal Suma : std_logic_vector(8 downto 0);
begin
```

- Sólo se declara la señal Estado_actual.

4. Descripción FSMD en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

- Descripción como FSMD Mealy (2).
 - Process del Registro de Estado.



- Se describe la secuencia de estados de la FSM.
- En cada estado de la FSM se indica la microoperación que debe realizarse, y por tanto, el funcionamiento de los registros. **Temp ← -1_{C2}**

- Microoperaciones:
 - Resta ← X
 - Resul ← 0

- Por tanto, la FSM describe la Ruta de Control y la de Datos.
- Por ello, se denomina FSMD.

```
process(Clk, reset)
begin
  If reset = '1' then
    Estado_actual <= Espera;
    Resul_temp <= (others => '0');
    Temp <= (others => '1');
  Elsif Clk'event and Clk = '1' then
    case Estado_actual is
      when Espera =>
        Temp <= (others => '1');
        If Inicio = '1' then
          Estado_actual <= Carga;
        End if;
      when Carga =>
        Resta <= X;
        Resul_temp <= (others => '0');
        Estado_actual <= Espera_resta;
    end case;
  end if;
end process;
```

4. Descripción FSMD en VHDL.

Descripción VHDL la Unidad de Cálculo de la Raíz Cuadrada

- Descripción como FSMD Mealy (3).
 - Process del Registro de Estado.

- Microoperaciones:
 - $Resta \leftarrow Resta + Temp$
 - $Resul \leftarrow Resul + 1$
 - $Temp \leftarrow Temp + -2_{C2}$

```
when Espera_resta =>
  If Suma(8) = '1' then
    Resta <= Suma(7 downto 0);
    Resul_temp <= resul_temp + 1;
    Temp <= Temp + "11111110";
    Estado_actual <= Espera_Resta;
  else
    Estado_actual <= Espera;
  end if;
when others =>
  Estado_actual <= Espera;
end case;
end if;
end process;
Suma <= ('0' & Resta) + ('0' & temp);
Resul <= Resul_temp;
Fin <= '1' when estado_actual = Espera_resta and Suma(8) = '0'
      else '0';
end Behavioral;
```

- En un process combinacional y/o con statements concurrentes de asignación de valores a señales se describen el resto de componentes combinacionales (sumador) y las salidas de control (Fin).

